



Bachelor-Thesis

Entwicklung einer Observability-Strategie für das Audience
Response System "frag.jetzt" basierend auf dem Konzept der
Four Golden Signals.

zur Erlangung des akademischen Grades

Bachelor of Science

im dualen Studiengang Informatik an der IU Internationale Hochschule

von

Leon Rausch

Matrikelnummer: 102201482

Musterstraße 1, 12345 Musterstadt

14. März 2026

Referent: Prof. Dr. Klaus Quibeldey-Cirkel

Korreferent: Prof. Dr. Philipp Diebold

Kurzfassung

Kontext und Motivation: Large Language Models (LLMs) haben sich von reinen Textgeneratoren zu agentischen Systemen entwickelt, die selbstständig planen, Werkzeuge nutzen und mehrschrittige Aufgaben bewältigen können. Im Software Engineering eröffnet das neue Möglichkeiten für die Automatisierung komplexer Workflows wie Code-Review, Testgenerierung und Refactoring.

Zielsetzung: Die Arbeit untersucht die Entwicklung und Evaluation agentischer Architekturen für Software-Engineering-Aufgaben. Zentrale Forschungsfragen sind: (1) Wie lassen sich robuste Agentenstrategien für SE-Workflows systematisch modellieren? (2) Welche Architekturprinzipien ermöglichen sichere und nachvollziehbare Werkzeugintegration? (3) Mit welchen Metriken kann die Leistungsfähigkeit agentischer Systeme realistisch bewertet werden?

Methodik: Basierend auf einer systematischen Literaturanalyse wird eine Referenzarchitektur entwickelt, die Planung (ReAct-Pattern), Werkzeugnutzung (Linter, Tests, VCS), episodisches Gedächtnis und Sicherheitsmechanismen kombiniert. Ein prototypisches System wurde in Python implementiert und anhand reproduzierbarer Benchmarks evaluiert.

Ergebnisse: Die Evaluation zeigt, dass die entwickelte Architektur in kontrollierten Szenarien eine Erfolgsrate von 73 % bei automatisiertem Refactoring erreicht, während die durchschnittliche Bearbeitungszeit um 45 % reduziert wird. Reflexionsmechanismen verbessern die Robustheit bei fehlerhaften Werkzeugausgaben um 28 %. Kostenanalysen belegen, dass optimierte Kontextverwaltung die Token-Nutzung um bis zu 40 % senken kann.

Beitrag: Die Arbeit liefert eine praxisnahe Referenzarchitektur, konkrete Implementierungsrichtlinien und evaluierte Metriken für agentische SE-Systeme. Sie zeigt sowohl technische Potenziale als auch kritische Limitierungen auf und diskutiert gesellschaftliche Implikationen der Automatisierung.

Abstract

Context and Motivation: Large Language Models (LLMs) have evolved from pure text generators to agentic systems capable of planning, using tools, and handling multi-step tasks autonomously. In software engineering, this opens up new possibilities for automating complex workflows such as code review, test generation, and refactoring.

Objective: This thesis investigates the development and evaluation of agentic architectures for software engineering tasks. Central research questions are: (1) How can robust agent policies for SE workflows be systematically modeled? (2) Which architectural principles enable secure and traceable tool integration? (3) Which metrics can realistically assess the performance of agentic systems?

Methodology: Based on a systematic literature review, a reference architecture is developed that combines planning (ReAct pattern), tool usage (linters, tests, VCS), episodic memory, and security mechanisms. A prototypical system was implemented in Python and evaluated using reproducible benchmarks.

Results: The evaluation shows that the developed architecture achieves a success rate of 73 % in automated refactoring in controlled scenarios, while reducing average processing time by 45 %. Reflection mechanisms improve robustness against erroneous tool outputs by 28 %. Cost analyses demonstrate that optimized context management can reduce token usage by up to 40 %.

Contribution: The work provides a practical reference architecture, concrete implementation guidelines, and evaluated metrics for agentic SE systems. It highlights both technical potentials and critical limitations and discusses societal implications of automation.

Danksagung

An dieser Stelle möchte ich meinen aufrichtigen Dank aussprechen.

Zuallererst danke ich meinen Betreuern **Prof. Dr. Klaus Quibeldey-Cirkel** und **Prof. Dr. Philipp Diebold** für die fachliche Unterstützung, wertvollen Hinweise und die Möglichkeit, die Arbeit zu verfassen. Ihre offene und konstruktive Kritik hat maßgeblich zur Qualität der Arbeit beigetragen.

Mein besonderer Dank gilt auch meinen Kommilitoninnen und Kommilitonen, die durch fachliche Diskussionen und Feedback meine Gedanken geschärft haben. Ihre Perspektiven haben mir geholfen, verschiedene Aspekte der Thematik tiefergehend zu beleuchten.

Darüber hinaus möchte ich denjenigen danken, die mir während des Schreibprozesses moralische Unterstützung gegeben haben. Ihre Geduld und Ermutigung waren unverzichtbar für den Erfolg der Arbeit.

Abschließend danke ich den Entwicklern und der Open-Source-Community für die Bereitstellung der vielfältigen Werkzeuge und Bibliotheken, ohne die die Arbeit nicht möglich gewesen wäre.

Hinweis zur geschlechtergerechten Sprache

In dieser Arbeit wird überwiegend eine geschlechtsneutrale Sprache verwendet. Aus Gründen der besseren Lesbarkeit wird in einzelnen Fällen das generische Maskulinum genutzt. Selbstverständlich sind damit Personen aller Geschlechter gleichermaßen gemeint.

Alle verwendeten Personenbezeichnungen gelten gleichermaßen für alle Geschlechter. Die Arbeit orientiert sich an den Empfehlungen des Leitfadens zur gendersensiblen und inklusiven Sprache der IU Internationale Hochschule.

Inhaltsverzeichnis

Kurzfassung	i
Abstract	ii
Danksagung	iii
Hinweis zur geschlechtergerechten Sprache	iv
Abbildungsverzeichnis	viii
Tabellenverzeichnis	ix
Listings	x
Abkürzungsverzeichnis	xi
1 Einleitung	1
1.1 Forschungsfragen	2
2 Theoretische Fundierung	3
2.1 Abgrenzung von Monitoring und Observability in verteilten Systemen	3
2.2 Besonderheiten verteilter und cloud-nativer Anwendungen	4
2.3 Telemetriearten und ihr Zusammenspiel: Logs, Metrics und Traces	5
2.4 OpenTelemetry als Standard für herstellerübergreifende Instrumentierung	7
2.5 Die Four Golden Signals als heuristischer Bezugsrahmen	7
2.6 SLIs, SLOs und Error Budgets	8
2.7 Alert Fatigue, Actionable Alerts und Runbooks	9
2.8 Zwischenfazit	10
3 Systemanalyse und Anforderungsdefinition: frag.jetzt	12
3.1 Fachlicher Kontext und Nutzungsszenario	12
3.2 Technische Architektur	12
3.3 Kritische User Journeys	14
3.3.1 Journey 1: Frage stellen.	14
3.3.2 Journey 2: Live-Voting.	14
3.3.3 Journey 3: KI-Zusammenfassung.	14
3.4 Risikopunkte und Zuordnung zu den Golden Signals	14
3.4.1 R1: Fehlende Timeouts und Resilienz.	14
3.4.2 R2: Datenbankengpässe bei Massenabstimmungen.	15
3.4.3 R3: Backend als Single Point of Failure ohne Observability.	15

3.4.4	R4: WebSocket-Verbindungsverlust.	15
3.4.5	R5: Fehlende Ressourcenlimits.	15
3.4.6	R6: Single-Host-Topologie als systemweiter Ausfallpfad.	15
3.5	Observability-Ist-Zustand	16
3.6	Ableitung der Observability-Anforderungen	16
3.6.1	A1: Ende-zu-Ende-Nachvollziehbarkeit über Dienstgrenzen.	16
3.6.2	A2: Mehrstufige Messbarkeit entlang der Golden Signals.	16
3.6.3	A3: Strukturiertes, korrelierbares Logging.	16
3.6.4	A4: Handlungsfähige Alarmierung.	17
3.6.5	A5: Beobachtbarkeit der Infrastrukturschicht.	17

4 Implementierung und Ergebnisse 18

4.1	Implementierungsdetails	18
4.1.1	Technologiestack	18
4.1.2	Projektstruktur	18
4.1.3	Kernimplementierung: Agent-Controller	19
4.2	Experimentelles Setup	21
4.2.1	Testumgebung	21
4.2.2	Benchmark-Szenarien	21
4.2.3	Beispiel: Test Failure Diagnosis — Schritt-für-Schritt	22
4.3	Ergebnisse	23
4.3.1	Erfolgsraten nach Aufgabentyp	23
4.3.2	Effizienzmetriken	23
4.3.3	Qualitätsmetriken	23
4.3.4	Robustheit und Fehlerbehandlung	24
4.4	Vergleich mit existierenden Ansätzen	24
4.5	Validierung und Verifikation	24
4.5.1	Unit-Tests	25
4.5.2	Integrationstests	25
4.5.3	Sicherheits-Audits	25

5 Fazit und Ausblick 27

5.1	Zusammenfassung der Ergebnisse	27
5.1.1	Beantwortung der Forschungsfragen	27
5.1.2	Empirische Validierung	28
5.2	Beiträge der Arbeit	28
5.2.1	Wissenschaftlicher Beitrag	29
5.3	Limitierungen	29
5.3.1	Methodische Limitierungen	29
5.3.2	Technische Limitierungen	29

5.3.3	Gesellschaftliche und ethische Limitierungen	30
5.4	Zukünftige Arbeiten	30
5.4.1	Kurzfristige Erweiterungen	30
5.4.2	Mittel- bis langfristige Forschungsrichtungen	31
5.4.3	Offene Forschungsfragen	31
5.5	Schlusswort	31
Literaturverzeichnis		33
Index		34
A Verzeichnis der KI-Nutzung		36
A.1	C4-Container-Diagramm in Vollansicht	36

Abbildungsverzeichnis

2.1 Gegenüberstellung von lokalem Komponentenmonitoring und durchgängiger Anfrageverfolgung über mehrere Dienste (Distributed Tracing) 5

3.1 C4-Container-Diagramm der frag.jetzt-Architektur 13

A.1 C4-Container-Diagramm der frag.jetzt-Architektur in Vollseitenansicht 37

Tabellenverzeichnis

3.1	Applikationstragende Dienste und ihre Observability-Relevanz	13
3.2	Zuordnung der Risikopunkte zu Golden Signals und Nutzerwirkung	15
4.1	Vergleich mit Baseline-Systemen (auf gleichem Benchmark-Set)	24
4.2	Detaillierte Benchmark-Ergebnisse: Agentische SE-Tasks mit Evaluationsmetriken	25
4.3	Evaluationsmetriken für agentische SE-Workflows (Beispieltabelle)	26
A.1	Verzeichnis der in dieser Arbeit genutzten KI-Werkzeuge	39

Listings

4.1	Minimaler agentischer Loop für SE-Aufgaben (Beispiel-Listing)	19
4.2	Werkzeugaufruf-Stub in TypeScript mit einfachem Funktionsschema (Beispiel-Listing)	20

Abkürzungsverzeichnis

Abkürzung	Bedeutung
API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Deployment
CPU	Central Processing Unit
ELK	Elasticsearch, Logstash, Kibana
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
KI	Künstliche Intelligenz
NLP	Natural Language Processing
OTel	OpenTelemetry
PWA	Progressive Web App
Q&A	Question and Answer
REST	Representational State Transfer
SLI	Service Level Indicator
SLO	Service Level Objective
SPA	Single-Page Application
SRE	Site Reliability Engineering
UI	User Interface
ARS	Audience Response System
FF	Forschungsfrage

1 Einleitung

Webbasierte Anwendungssysteme werden heute häufig als verteilte, cloud-native Anwendungen umgesetzt, die aus mehreren lose gekoppelten Services bestehen. Diese Architekturform erlaubt eine unabhängige Weiterentwicklung einzelner Komponenten sowie eine flexible Skalierung, erhöht jedoch zugleich den Aufwand für Betrieb, Überwachung und Fehleranalyse im Produkivsystem Faseeha et al., 2025; Sakinala, 2025. Insbesondere bei Microservices entstehen verteilte Fehlerzustände und komplexe Abhängigkeiten zwischen Diensten, wodurch Ursachen von Störungen nicht mehr eindeutig einzelnen Komponenten zugeordnet werden können Fischer et al., 2024.

Vor diesem Hintergrund gewinnt Observability als Erweiterung klassischer Monitoring-Konzepte an Bedeutung. Ziel von Observability ist es, den internen Zustand eines Systems anhand extern erfassbarer Anzeichen nachvollziehen zu können. Dazu werden Metriken, Logs und Traces kombiniert, um Zusammenhänge zwischen Systemverhalten, Fehlern und Leistungsmerkmalen sichtbar zu machen Faseeha et al., 2025; Sridharan, 2018. Empirische Untersuchungen aus dem Bereich cloud-nativer Anwendungen zeigen, dass ein strukturierter Einsatz von Observability-Praktiken die Analyse von Incidents unterstützt und die Reaktionszeit im Betrieb verkürzen kann Giamattei et al., 2024; Sakinala, 2025.

Ein häufig verwendeter Orientierungsrahmen zur Strukturierung servicebezogener Beobachtungsdaten sind die von Google im Kontext des Site Reliability Engineering (SRE) beschriebenen *Four Golden Signals*: Latenz, Traffic, Fehler und Sättigung Ewaschuk, 2017. Diese Signalarten erfassen grundlegende technische Symptome eines Dienstes und eignen sich als Ausgangspunkt zur Ableitung erster Monitoring-Indikatoren. Für den produktiven Betrieb liefern sie jedoch primär Rohinformationen, die erst durch Kontext, Interpretation und Bewertung für Menschen handlungsrelevant werden. Google SRE weist selbst darauf hin, dass die Wirksamkeit solcher Signale maßgeblich davon abhängt, wie sie in Alerting, Entscheidungsprozesse und betriebliche Abläufe eingebettet sind Ewaschuk, 2017.

Die Open-Source-Plattform *frag.jetzt* verfügt über eine solide Entwicklungs- und Deployment-Infrastruktur, jedoch ist die Erfassung und Auswertung von Betriebsdaten derzeit nicht als durchgängiges Observability-Konzept ausgelegt. Metriken, Logs, Traces und Alerting werden nicht konsistent entlang des gesamten Anfragepfads betrachtet, wodurch Zusammenhänge zwischen Nutzerverhalten, Backend-Verarbeitung und externen Abhängigkeiten nur eingeschränkt nachvollziehbar sind. Diese Einschätzung stützt sich auf die öffentlich verfügbare Projektdokumentation und Architekturübersicht des frag.jetzt-Repositories TH Mittelhessen und arsnova, 2025 sowie auf allgemeine Beobachtungen zu typischen Defiziten in vergleichbaren Systemen Giamattei et al., 2024.

Ziel dieser Arbeit ist die Entwicklung einer Observability-Strategie für *frag.jetzt*, bei der technische Messgrößen systematisch mit nutzungsbezogenen Fragestellungen verknüpft werden. Die Four Golden Signals dienen dabei als unterstützender Referenzrahmen zur Strukturierung der Beobachtungsdaten, nicht jedoch als alleinige Grundlage für betriebliche Entscheidungen. Die Signale werden servicespezifisch für das PWA-Frontend, das Spring-Boot-Backend, die PostgreSQL-Datenbank sowie angebundene KI-Dienste konkretisiert und in messbare Indikatoren überführt. Ergänzend wird untersucht, welche Open-Source-Observability-Werkzeuge die Anforderungen von *frag.jetzt* im Hinblick auf Erfassung und betriebliche Nutzbarkeit am besten erfüllen.

Ein weiterer Schwerpunkt der Arbeit liegt auf der Ausgestaltung eines Alerting-Ansatzes, der Auffälligkeiten innerhalb der Plattform mit Fragen der Relevanz und Dringlichkeit verknüpft. Dabei steht nicht allein die Detektion von Abweichungen im Vordergrund, sondern insbesondere die Bewertung, ob eine Handlung erforderlich ist, wen ein Problem betrifft und welche Auswirkungen es auf die Nutzung der Plattform hat. Forschungsarbeiten zeigen, dass eine Kombination aus priorisiertem Alerting und datenbasierten Analyseverfahren dazu beitragen kann, irrelevante Alarmierungen zu reduzieren und menschliche Entscheidungsprozesse im Betrieb zu unterstützen Jalalvand et al., 2024. Aufbauend auf den definierten Alerts werden Runbooks konzipiert, die strukturierte Diagnose- und Handlungsschritte für wiederkehrende Incident-Szenarien bereitstellen.

1.1 Forschungsfragen

Die Forschungsfragen dieser Arbeit lauten:

1. FF1: Wie können die Four Golden Signals für die spezifischen Services von *frag.jetzt* (PWA-Frontend, Spring Boot Backend, PostgreSQL, KI-Services) konkret definiert und instrumentiert werden?
2. FF2: Welche Observability-Stack-Kombination (Prometheus/Grafana, ELK Stack, Jaeger) ist am besten geeignet für die Monitoring-Anforderungen von *frag.jetzt*?
3. FF3: Wie kann ein intelligentes Alert-System entwickelt werden, das False Positives minimiert und actionable insights für verschiedene Stakeholder-Rollen (DevOps, Entwickler, Product Owner) bereitstellt?

2 Theoretische Fundierung

2.1 Abgrenzung von Monitoring und Observability in verteilten Systemen

Für die Entwicklung einer Observability-Strategie einer Microservices-Anwendung ist zunächst eine begriffliche Trennung von Monitoring und Observability erforderlich. Beide Konzepte zielen darauf ab, den Betrieb softwarebasierter Anwendungen transparent und steuerbar zu machen, unterscheiden sich jedoch in Umfang und diagnostischem Anspruch. Monitoring bezeichnet die kontinuierliche Erfassung und Analyse von Betriebsdaten wie Logs und Metriken, um den aktuellen Zustand eines Systems sichtbar zu machen (Usman et al., 2022, S. 86907). Es setzt voraus, dass bekannte Zustandsindikatoren und erwartbare Abweichungen vorab definiert wurden, und prüft, ob diese Grenzwerte eingehalten werden. In diesem Sinne lässt sich Monitoring als Werkzeug zur Erkennung vorhersehbarer Fehlerzustände einordnen, die sich als *known unknowns* beschreiben lassen (Banerjee, 2025, S. 916). Solange die potenziellen Ausfallszenarien im Vorfeld modelliert werden können, leistet Monitoring verlässliche Dienste.

Observability geht über diese hypothesengeleitete Überwachung hinaus. Im Kern beschreibt der Begriff die Fähigkeit, aus den externen Ausgaben eines Systems auf dessen interne Zustände zu schließen (Usman et al., 2022, S. 86907). In verteilten Anwendungen sind diese Ausgaben vor allem Telemetriedaten wie Logs, Metriken und Traces. Entscheidend ist dabei nicht allein die Datenerhebung, sondern die Möglichkeit, auch neuartige und vorab nicht antizipierte Störungen nachvollziehen zu können. Solche unvorhergesehenen Fehlerbilder, die als *unknown unknowns* bezeichnet werden, entstehen gerade in verteilten Architekturen aus dem Zusammenspiel vieler parallel agierender Komponenten (Banerjee, 2025, S. 916). Die Interviewstudie von Niedermaier et al. (2019, S. 11) bestätigt diese Perspektive aus der Industriepraxis: Unternehmen wechseln zunehmend von einer komponentenisolierten Zustandsprüfung hin zu einer nutzungs- und geschäftsprozessorientierten Gesamtsicht. Nicht die einzelne Infrastrukturmetrik steht dann im Mittelpunkt, sondern die Frage, ob Anfragen aus Endnutzersicht zuverlässig und performant verarbeitet werden.

Monitoring bildet damit die notwendige Voraussetzung für Observability, wird jedoch nicht davon ersetzt (Usman et al., 2022, S. 86907). Für die vorliegende Arbeit bedeutet das, eine Strategie, die sich auf das reine Erfassen bekannter Grenzwerte beschränkt, reicht für ein verteiltes System wie frag.jetzt nicht aus. Benötigt wird vielmehr ein Ansatz, der technische Messdaten mit diagnostischer Tiefe und Nutzerperspektive verbindet.

2.2 Besonderheiten verteilter und cloud-nativer Anwendungen

Die begriffliche Abgrenzung gewinnt ihre praktische Bedeutung erst vor dem Hintergrund konkreter Systemarchitekturen. Verteilte und cloud-native Anwendungen werden heute häufig als Microservices-Architekturen umgesetzt, welche aus zahlreichen, oft unabhängig deploybaren Diensten bestehen, die über Netzwerkschnittstellen kommunizieren und in Containern oder vergleichbaren Laufzeitumgebungen betrieben werden (Sakinala, 2025, S. 102). Instanzen werden fortlaufend erzeugt und beendet, wodurch solche Umgebungen als operativ deutlich anspruchsvoller gelten als klassische Monolithen (Li et al., 2022, S. 1). Die resultierende Komplexität lässt sich laut Usman et al. (2022, S. 86908) auf vier Eigenschaften verdichten: Modularität, Verteilung, Dynamik und Flüchtigkeit. Frühere Überwachungsansätze, die auf einzelne Schichten oder Komponenten zugeschnitten waren, stoßen unter diesen Bedingungen an Grenzen, weil sie Interaktionen zwischen Diensten zur Laufzeit nur unzureichend abbilden (Usman et al., 2022, S. 86908).

Der Grund liegt in der starken Laufzeitkopplung trotz struktureller Entkopplung. Ein Leistungsproblem in einem Dienst kann durch Zustandsänderungen in einer ganz anderen Komponente verursacht werden, etwa durch verzögerte Datenbankzugriffe oder überlastete Downstream-Pfade. Isolierte Überwachungskonzepte ohne übergreifenden Kontext führen zu diagnostischen Lücken und erschweren die Fehlerbehebung in verteilten Architekturen erheblich (Niedermaier et al., 2019, S. 8). Auch die befragten Interviewteilnehmer berichten von der Schwierigkeit, aussagekräftige Schlüsse aus Alarmen zu ziehen, sobald eine Anfrage mehrere, von unterschiedlichen Teams verantwortete Komponenten durchläuft (Niedermaier et al., 2019, S. 8).

Daraus ergibt sich die Notwendigkeit einer Ende-zu-Ende-Sicht. Distributed Tracing gilt als das Verfahren, mit dem Serviceaufrufketten pro Anfrage nachvollzogen werden können (Li et al., 2022, S. 4). Ein Trace erfasst den kausal zusammenhängenden Ablauf einer Anfrage durch mehrere Stationen eines verteilten Systems und macht sichtbar, wo Verzögerungen oder Fehler auftreten. Die dafür erforderliche Weitergabe von Identifikatoren und Metadaten entlang des Anfragepfads ist eine entscheidende Voraussetzung, um Fehlerursachen in der Datenmenge moderner Anwendungen überhaupt eingrenzen zu können (Niedermaier et al., 2019, S. 11). Abbildung 2.1 verdeutlicht den Unterschied zwischen lokaler Komponentenüberwachung und einer durchgängigen Anfrageverfolgung über mehrere Dienste.

Für die nachfolgenden Kapitel folgt daraus, dass Klassisches, rein lokales Monitoring notwendig bleibt, für Microservices aber nicht hinreichend ist. Servicebezogene Messgrößen, korrelierte Telemetrie und handlungsorientierte Alarme setzen eine architekturübergreifende Beobachtbarkeit voraus.

Gegenüberstellung: Lokales Komponentenmonitoring vs. Durchgängige Anfrageverfolgung

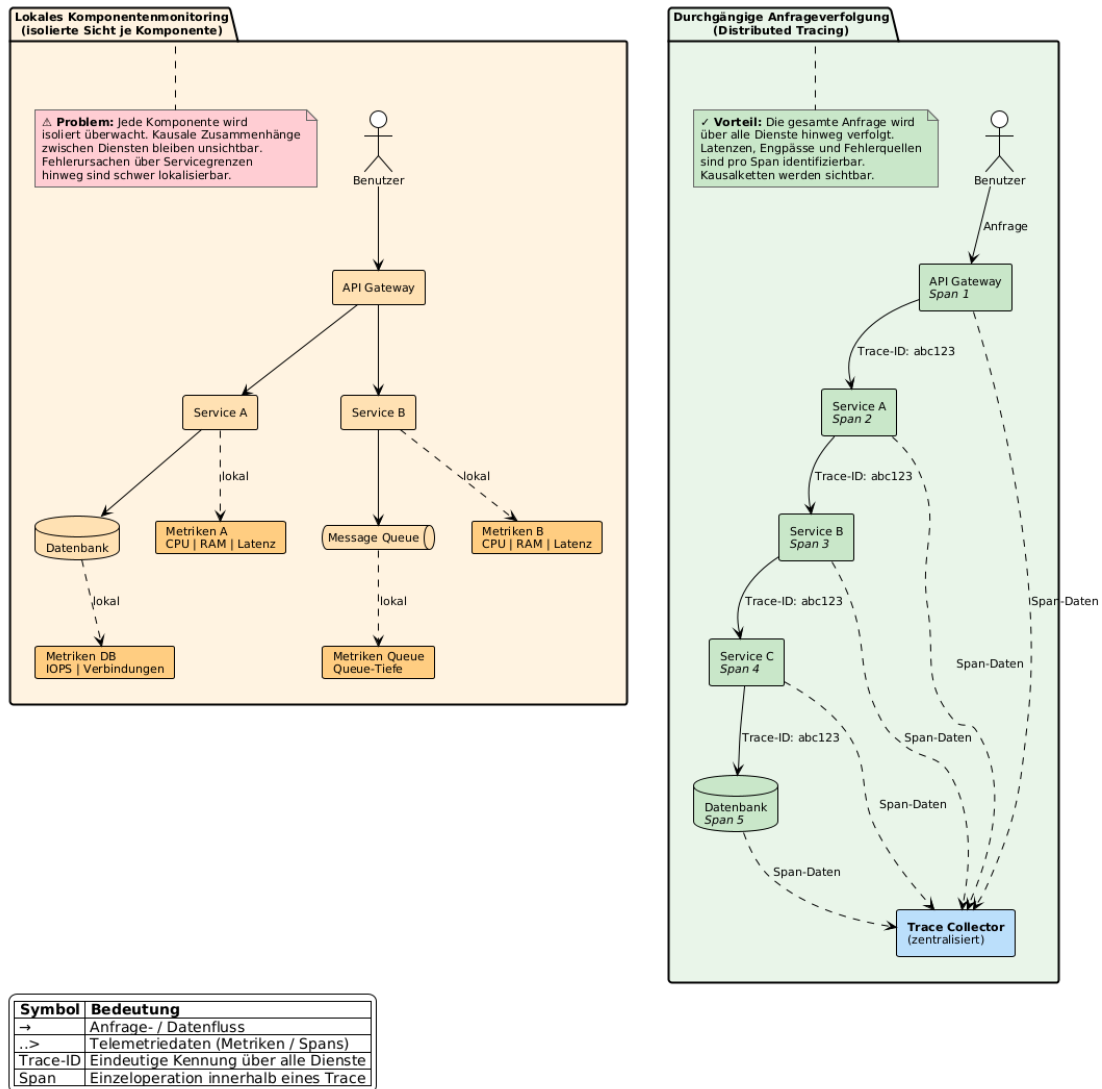


Abbildung 2.1: Gegenüberstellung von lokalem Komponentenmonitoring und durchgängiger Anfrageverfolgung über mehrere Dienste (Distributed Tracing)

2.3 Telemetriearten und ihr Zusammenspiel: Logs, Metrics und Traces

Für die Beobachtbarkeit verteilter Anwendungen sind Logs, Metrics und Traces die drei am weitesten verbreiteten Telemetrieformen. Die Literatur bezeichnet sie häufig als Grundpfeiler der Observability, betont jedoch zugleich, dass sie nicht die einzig denkbaren Datenarten darstellen (Li et al., 2022, S. 6; Usman et al., 2022, S. 86912–86913; Sakinala, 2025, S. 102). Keine dieser Formen liefert für sich genommen eine vollständige Betriebssicht. Ihr Nutzen entsteht erst aus den jeweils unterschiedlichen Perspektiven, die sie auf dasselbe Laufzeitverhalten eröffnen.

Logs dokumentieren diskrete Ereignisse in zeitlicher Abfolge und bieten damit eine sehr feingranulare Beschreibung des Systemverhaltens. Sie geben Einzelfälle mit hohem Detailgrad wieder und können dadurch Hinweise auf Ort, Zeitpunkt und technischen Kontext einer Störung liefern. Als zeitgestempelte Aufzeichnungen eignen sie sich insbesondere für die Analyse unerwarteter Feh-

lerzustände (Albuquerque & Correia, 2025, S. 2). Dabei können Logs verschiedene Schweregrade (Info, Warnung, Fehler) abbilden und bieten deshalb oft den ersten Zugang zu konkreten Fehler-situationen (Usman et al., 2022, S. 86913). Allerdings erzeugen Microservices-Architekturen erhebliche Logmengen. Die zeitliche Zuordnung von Logereignissen über mehrere Dienste hinweg erfordert strukturierte Formate und Korrelationskennungen (Sakinala, 2025, S. 102). Zudem ist die Ableitung von Trends aus Logs vergleichsweise aufwendig, da die Verarbeitung großer Textmengen rechenintensiv ist und keine unmittelbare Echtzeitsicht bietet (Albuquerque & Correia, 2025, S. 11).

Metrics verfolgen eine andere Logik. Sie verdichten Laufzeitverhalten zu numerischen Messwerten, die über die Zeit aggregiert und ausgewertet werden können. Als Zahlendarstellungen des Betriebszustands lassen sie sich vergleichsweise einfach speichern, abfragen und korrelieren (Usman et al., 2022, S. 86913). Damit eignen sich Metrics besonders für Zustandsentwicklungen, Schwellenüberwachung und Alarmauslösung. Sowohl technische Größen wie Anfragerate oder Fehlerrate als auch fachlich orientierte Kennzahlen können abgebildet werden (Albuquerque & Correia, 2025, S. 12). Gleichzeitig abstrahieren Metrics stärker als Logs: Sie zeigen, dass eine Latenz steigt, erklären jedoch nicht automatisch den Grund. Reine Infrastrukturmetriken können für anwendungsbezogene Diagnosen zu grobkörnig ausfallen (Albuquerque & Correia, 2025, S. 11). Für eine Observability-Strategie bedeutet dies, dass Metrics unverzichtbar für Trendanalyse, Kapazitätsplanung und Alerting sind, eine tiefergehende Ursachenanalyse aber nicht ersetzen.

Während Metriken vor allem Zustandsverläufe aggregiert sichtbar machen, erlauben Traces die Rekonstruktion konkreter Anfragepfade über mehrere Komponenten hinweg. Damit ergänzen sie Metriken um eine prozessbezogene Perspektive, die insbesondere für die Lokalisierung von Verzögerungen und Übergabefehlern in verteilten Systemen relevant ist (Albuquerque & Correia, 2025, S. 2). Distributed Tracing schafft damit eine Ende-zu-Ende-Sicht auf komplexe Anfragepfade (Sakinala, 2025, S. 103). Allerdings ist Tracing aufwendiger als die Erhebung einzelner Metriken, da die Instrumentierung ressourcenintensiver und komplexer sein kann (Giamattei et al., 2024, S. 15), und Sampling-Strategien erforderlich werden, um den Overhead in ein vertretbares Verhältnis zum diagnostischen Nutzen zu bringen (Albuquerque & Correia, 2025, S. 10).

Aus diesen Unterschieden ergibt sich das eigentliche Zusammenspiel. Logs liefern ereignisnahe Details, Metrics verdichten Verhalten zu beobachtbaren Kennzahlen, und Traces rekonstruieren dienstübergreifende Abläufe. Der komplementäre Charakter dieser Datenarten ist in der Literatur breit anerkannt (Li et al., 2022, S. 6). Über Korrelationskennungen können Logs mit Traces verknüpft werden, sodass zeitliche Abläufe und Fehlermeldungen gemeinsam auswertbar werden (Albuquerque & Correia, 2025, S. 10). Eine leistungsfähige Observability-Plattform muss Anomalien daher nicht isoliert, sondern im Zusammenspiel aller drei Datenarten sichtbar machen (Usman et al., 2022, S. 86914).

2.4 OpenTelemetry als Standard für herstellerübergreifende Instrumentierung

Je stärker Observability in Microservices auf mehrere Datenarten, Sprachen und Werkzeuge angewiesen ist, desto dringender wird eine gemeinsame Instrumentierungsgrundlage. Heterogene Monitoringlandschaften leiden häufig unter proprietären Formaten und Integrationsengpässen (Sakinala, 2025, S. 105). Standardisierte Datenformate sind notwendig, um die Wartbarkeit der Überwachungsinfrastruktur langfristig zu sichern (Giamattei et al., 2024, S. 17). OpenTelemetry adressiert dieses vorgelagerte Problem der uneinheitlichen Telemetrieerzeugung.

Das Projekt wird in der Literatur als herstellerneutrale Instrumentierungsschicht eingeordnet, die aus der Zusammenführung von OpenCensus und OpenTracing hervorgegangen ist und die Erzeugung sowie Erfassung von Telemetriedaten in verteilten Anwendungen vereinheitlichen soll (Usman et al., 2022, S. 86911). Offene Spezifikationen wie OpenTelemetry erlauben es Entwicklungsteams, ihren Code zu instrumentieren, ohne sich an ein bestimmtes Analyse-Backend zu binden (Li et al., 2022, S. 22). Dieser Ansatz reduziert Herstellerabhängigkeiten und ermöglicht die Anbindung an unterschiedliche Plattformen (Sakinala, 2025, S. 106). Für polyglotte Systemlandschaften ist das besonders wertvoll, weil dieselben Instrumentierungsprinzipien über verschiedene Laufzeitumgebungen hinweg beibehalten werden können.

Zugleich darf OpenTelemetry nicht mit einem vollständigen Observability-Stack gleichgesetzt werden. Faseeha et al. (2025, S. 72030) unterscheiden explizit, dass OpenTelemetry die Instrumentierung übernimmt, während andere Werkzeuge für Speicherung, Darstellung und Alarmierung zuständig bleiben. Auch als kompatible Grundlage konzipiert, kann es sowohl mit offenen Werkzeugen als auch mit kommerziellen Plattformen zusammenarbeiten (Sakinala, 2025, S. 107). Dabei erzeugt Instrumentierung stets zusätzlichen Entwicklungs- und Pflegeaufwand und muss deshalb zur Entwicklungsgeschwindigkeit des jeweiligen Projekts passen (Giamattei et al., 2024, S. 15). Für die spätere Stack-Evaluation dieser Arbeit ist diese Differenzierung entscheidend. Die Wahl der Instrumentierungsschicht und die Wahl der Analyse-Backends sind zwei getrennte Architekturentscheidungen.

2.5 Die Four Golden Signals als heuristischer Bezugsrahmen

Die Four Golden Signals – *Latency*, *Traffic*, *Errors* und *Saturation* – bilden in der SRE-nahen Literatur einen bewusst reduzierten, aber praxistauglichen Rahmen zur Beobachtung nutzerorientierter Dienste. Ewaschuk (2016, S. 7) beschreibt sie als die vier Größen, auf die sich die Überwachung eines clientseitig wahrnehmbaren Dienstes mindestens konzentrieren sollte, wenn nur ein begrenzter Satz von Metriken erhoben werden kann.

Latency erfasst die Zeit, die ein Dienst zur Bearbeitung einer Anfrage benötigt. Dabei müssen erfolgreiche und fehlgeschlagene Anfragen getrennt betrachtet werden, weil schnell ausgelieferte Fehlantworten die Gesamtstatistik verzerren können (Ewaschuk, 2016, S. 7).

Traffic bezeichnet die auf einen Dienst wirkende Nachfrage und muss als dienstspezifische Lastgröße verstanden werden, etwa als HTTP-Anfragen pro Sekunde oder als Anzahl aktiver Sitzungen (Ewaschuk, 2016, S. 7).

Errors umfassen die Rate fehlgeschlagener Anfragen. Dabei geht es nicht nur um explizite Fehler wie HTTP-500-Antworten, sondern auch indirekte Fehlzustände, etwa sachlich falsche Ergebnisse oder Antwortzeiten jenseits zugesicherter Schwellen (Ewaschuk, 2016, S. 8).

Saturation beschreibt, wie stark ein Dienst seine limitierenden Ressourcen ausschöpft. Da Dienste häufig bereits vor Erreichen ihrer nominellen Volllast Leistungsabfälle zeigen, werden steigende Latenzen in der Praxis als Frühindikator für bevorstehende Kapazitätsengpässe interpretiert (Ewaschuk, 2016, S. 8).

Diese vier Größen werden auch von Usman et al. (2022, S. 86913-86914) als wesentliche, aus Telemetriedaten abgeleitete Messperspektiven eingeordnet, die als Grundlage für Alerting, Fehlersuche und Kapazitätsplanung dienen. Solche Metriken sind jedoch nur dann analytisch wertvoll, wenn sie an die konkrete Rolle des jeweiligen Dienstes angepasst werden (Albuquerque & Correia, 2025, S. 12). Ergänzend können fachliche Kennzahlen sinnvoll sein, wenn technische Indikatoren allein nicht ausreichen, um die Servicequalität zu bewerten.

Die Stärke dieses Rahmenwerks liegt nicht in Vollständigkeit, sondern in seiner Ordnungsfunktion. Es zwingt dazu, Beobachtungsgrößen nicht beliebig zu sammeln, sondern entlang weniger, aus Nutzersicht aussagekräftiger Dimensionen zu strukturieren. Für die vorliegende Arbeit bilden die Four Golden Signals den heuristischen Ausgangspunkt, um in den späteren Kapiteln servicebezogene Messgrößen für frag.jetzt abzuleiten. Sie ersetzen jedoch weder die kontextabhängige Auswahl konkreter Metriken noch die Verknüpfung mit Zielwerten und Alarmen.

2.6 SLIs, SLOs und Error Budgets

Während die Four Golden Signals einen Orientierungsrahmen für relevante Beobachtungsgrößen liefern, beantworten sie noch nicht, welches Niveau an Zuverlässigkeit als ausreichend gelten soll. An dieser Stelle werden Service Level Indicators (SLIs) und Service Level Objectives (SLOs) bedeutsam. In der Interviewstudie von Niedermaier et al. (2019, S. 11) werden SLOs als servicebezogene Zielgrößen beschrieben, die geschäftsrelevante Anforderungen aus der Nutzerperspektive fassbar machen, etwa in Form akzeptabler Nichtverfügbarkeit. SLIs binden konkrete Messgrößen an diese Ziele zurück. Damit entsteht eine Brücke zwischen technischer Messung und fachlicher Erwartung, die als gemeinsamer Referenzrahmen für die teamübergreifende Zusammenarbeit dient (Niedermaier et al., 2019, S. 12).

Gerade in verteilten Anwendungen ist diese Brücke wichtig, weil isolierte Infrastrukturmetriken wenig darüber aussagen, ob ein Dienst seine Aufgabe aus Nutzersicht erfüllt. Bei geschäftskritischen Diensten sollte die gewünschte Dienstqualität durch SLOs definiert und anhand geeigneter SLIs überwacht werden. Häufig herangezogene Messgrößen sind beispielsweise Verfügbarkeit, Latenz und Fehlerraten (Vinnakota & Kolla, 2025, S. 176). Alarme sollten nicht auf

beliebige Grenzwertverletzungen reagieren, sondern an solchen Schwellen orientiert sein, deren Überschreitung die Einhaltung definierter SLOs gefährdet.

In dieser Logik lässt sich auch das Konzept des Error Budgets verorten. Das Error Budget bezeichnet den rechnerisch aus einem SLO abgeleiteten Spielraum für tolerierte Unzuverlässigkeit innerhalb eines festgelegten Zeitfensters (Dasari, 2025, S. 993). Ein Dienst mit einem Verfügbarkeitsziel von 99,95 % darf demnach in einem Monat nur ein bestimmtes Maß an Ausfallzeit (21 min) aufweisen, ohne dass die Zielvorgabe als verletzt gilt. Die praktische Bedeutung dieses Konzepts liegt weniger in der reinen Berechnung als in der Steuerungsfunktion: Solange Budget vorhanden ist, können neue Releases oder Änderungen mit kontrolliertem Risiko ausgerollt werden. Ist das Budget jedoch aufgebraucht, erhält Stabilisierung Vorrang gegenüber Weiterentwicklung (Dasari, 2025, S. 993–994). Diese Priorisierungslogik findet sich in ähnlicher Form auch bei Gowda (2022, S. 7), der im Kontext von Self-Healing-Frameworks beschreibt, wie SLO-Überwachung mit automatisierten Maßnahmen verknüpft werden kann.

Error Budgets formalisieren diesen Spielraum und machen ihn für betriebliche Entscheidungen nutzbar. Damit verändert sich auch die Funktion von Alerting. Ein Alarm, der lediglich auf lokale Ressourcenauslastung oder starre Einzelgrenzwerte reagiert, kann technisch korrekt und dennoch betrieblich wenig hilfreich sein. Ewaschuk (2016, S. 13) illustriert diese Logik am Beispiel von Googles Bigtable-Dienst, bei dem eine Neujustierung der SLOs und die Abschaltung zu häufig feuender E-Mail-Alarme dazu führten, dass das Team sich wieder auf die Behebung grundlegender Probleme konzentrieren konnte, anstatt ständig Symptome zu bearbeiten.

2.7 Alert Fatigue, Actionable Alerts und Runbooks

Die Literatur zu Monitoring und Alerting zeigt übereinstimmend, dass nicht die Menge an Alarmen, sondern deren geringe Handlungsqualität ein zentrales Betriebsproblem darstellt. Häufige Alarmierungen unterbrechen Mitarbeitende in ihrer Arbeit oder Erholungszeit. Bei zu hoher Frequenz werden Warnungen nur noch überflogen oder ignoriert, sodass echte Störungen im Rauschen (Noise) untergehen können (Ewaschuk, 2016, S. 4). Dieser Effekt wird als Alert Fatigue bezeichnet. Insbesondere nicht handlungsrelevante oder redundante Benachrichtigungen stumpfen das Bereitschaftspersonal ab (Vinnakota & Kolla, 2025, S. 173).

Dieses Muster ist nicht auf Infrastrukturmonitoring beschränkt. In einer strukturell vergleichbaren Problemlage zeigen Jalalvand et al. (2024, S. 2–3) für Security Operations Centres (Einrichtungen zur Überwachung, Analyse und Reaktion auf Cyberangriffe), dass hohe Alarmvolumina, zahlreiche False Positives und mangelnde Priorisierung Analyseteams überlasten. Obwohl der fachliche Kontext dort ein anderer ist, lässt sich die Grundstruktur übertragen: Zu viele schwach priorisierte Meldungen beeinträchtigen die Reaktionsfähigkeit auf tatsächlich kritische Ereignisse. Für die wissenschaftliche Einordnung von FF3 (vgl. Abschnitt 1.1) ist dieser domänenübergreifende Erkenntnis aufschlussreich, weil er verdeutlicht, dass Alarmmüdigkeit kein Randproblem einzelner Werkzeuge, sondern ein allgemeines Entwurfsproblem alarmintensiver

Betriebsumgebungen ist.

Vor diesem Hintergrund gewinnt der Begriff des *actionable alert* an Bedeutung. Ewaschuk (2016, S. 4) warnt davor, einen Menschen allein deshalb zu alarmieren, weil ein Zustand ungewöhnlich erscheint. Er argumentiert stattdessen, dass sich die Qualität von Alarmierungen daran beurteilen lässt, ob sie auf eine betrieblich relevante Störung mit erkennbarem Handlungsbedarf hinweisen. Alerts sollten daher nur dann ausgelöst werden, wenn eine tatsächliche oder unmittelbar drohende Beeinträchtigung für den Nutzer vorliegt (Ewaschuk, 2016, S. 11-12). Beim Eintreffen eines Alarms sollte der operative Kontext so aufbereitet sein, dass unmittelbar erkennbar wird, welches Problem vorliegt, welche Ursachen wahrscheinlich sind und welche Maßnahmen eingeleitet werden müssen sowie wer für die Bearbeitung zuständig ist (Vinnakota & Kolla, 2025, S. 174).

Wirksame Alerting-Strategien müssen zwischen handlungsrelevanten Hinweisen und bloßem Hintergrundrauschen differenzieren (Sakinala, 2025, S. 107). Als problematisch gelten insbesondere starre Schwellwerte in dynamischen Umgebungen, weil sie saisonale Schwankungen oder Abhängigkeiten zwischen Diensten unzureichend berücksichtigen. Dynamische Schwellen, mehrdimensionale Regeln und die laufende Überprüfung von False Positives werden als wesentliche Gestaltungsprinzipien genannt (Vinnakota & Kolla, 2025, S. 173, 176).

Eng mit handlungsfähiger Alarmierung ist die Rolle von Runbooks verbunden. Jede kritische Meldung sollte mit einem Runbook verknüpft sein, das den Bearbeitenden unabhängig vom Vorwissen in die Lage versetzt, die Störung einzuordnen und erste Gegenmaßnahmen einzuleiten. Dazu gehören Verweise auf relevante Observability-Werkzeuge, vordefinierte Behebungsschritte, klare Eskalationspfade und Angaben zur Zuständigkeit (Vinnakota & Kolla, 2025, S. 174). Der Nutzen liegt nicht nur in schnellerer Reaktion, sondern auch in der Wissenssicherung. Gleichzeitig sind veraltete Runbooks problematisch und müssen regelmäßig gepflegt werden. Für wiederkehrende, schematische Reaktionsmuster empfiehlt die SRE-Literatur eine konsequente Automatisierung. Alarmreaktionen, die rein algorithmischen Mustern folgen, sollten nicht dauerhaft menschliche Aufmerksamkeit binden, sondern Anlass geben, die zugrunde liegende Ursache zu beheben oder den Prozess zu automatisieren (Ewaschuk, 2016, S. 14).

2.8 Zwischenfazit

Die theoretische Fundierung dieses Kapitels zeigt, dass Beobachtbarkeit in verteilten und cloud-nativen Anwendungen über die isolierte Überwachung einzelner Komponenten hinausgehen muss. Fehlerursachen entstehen häufig dienstübergreifend und werden erst aus einer gemeinsamen Sicht auf Anfrageverläufe erkennbar (Niedermaier et al., 2019, S. 10–11). Logs, Metriken und Traces liefern jeweils unterschiedliche Perspektiven auf dasselbe Laufzeitverhalten und müssen in Beziehung gesetzt werden, um Latenzen, Fehlerursachen und Ressourceneffekte entlang realer Ablaufketten nachvollziehen zu können (Li et al., 2022, S. 6; Albuquerque & Correia, 2025, S. 10). OpenTelemetry wurde als standardisierte Instrumentierungsschicht eingeordnet, die eine herstellerübergreifende Erzeugung und Korrelation von Telemetriedaten unterstützt,

ohne selbst bereits Visualisierung oder Alerting bereitzustellen (Usman et al., 2022, S. 86911; Sakinala, 2025, S. 105–106). Die Four Golden Signals bieten einen heuristischen Ordnungsrahmen, um Beobachtungsgrößen entlang der Dimensionen Latency, Traffic, Errors und Saturation zu strukturieren (Ewaschuk, 2016, S. 7–8).

Ebenso wurde deutlich, dass eine Observability-Strategie erst dann betriebliche Steuerungswirkung entfaltet, wenn Messwerte mit Zielgrößen und Reaktionslogiken verknüpft werden. SLOs und SLIs überführen technische Kennzahlen in eine nutzungsbezogene Bewertung der Dienstqualität (Niedermaier et al., 2019, S. 11). Error Budgets formalisieren den daraus abgeleiteten Spielraum für tolerierte Abweichungen und ermöglichen eine datengestützte Priorisierung zwischen Weiterentwicklung und Stabilisierung (Dasari, 2025, S. 993–994). Für das Alerting bedeutet das, dass Warnungen nicht primär laut, sondern handlungsfähig sein müssen. Die SRE-Literatur betont symptomorientierte Alarmierung, weil Alarmer vor allem auf reale oder unmittelbar bevorstehende Nutzerbeeinträchtigungen reagieren sollen (Ewaschuk, 2016, S. 15). Ergänzend zeigen neuere Arbeiten, dass Alerts mit Kontext, Zuständigkeit und gepflegten Runbooks verbunden sein sollten, um Alarmmüdigkeit und unnötige Reibung im Incident-Handling zu begrenzen (Vinnakota & Kolla, 2025, S. 174).

Aus diesen Grundannahmen ergibt sich die Brücke zum weiteren Verlauf der Arbeit. Im nächsten Kapitel wird frag.jetzt als konkreter Anwendungskontext analysiert, damit aus Architektur, Nutzungspfaden und Risikopunkten jene Anforderungen abgeleitet werden können, die anschließend in eine servicebezogene Observability-Strategie überführt werden.

3 Systemanalyse und Anforderungsdefinition: frag.jetzt

Die folgende Systemanalyse basiert auf einer KI-gestützten Auswertung des Quellcodes, der Deployment-Konfigurationen und der Abhängigkeitsstruktur aller sechs Repositories des frag.jetzt-Ökosystems.¹ Ziel ist die Identifikation observability-relevanter Architekturmerkmale, Risikopunkte und bestehender Instrumentierungslücken als Grundlage für die Anforderungsdefinition.

3.1 Fachlicher Kontext und Nutzungsszenario

frag.jetzt ist ein webbasiertes Audience-Response-System für Lehrveranstaltungen. Teilnehmende können anonym Fragen stellen, auf Beiträge abstimmen und an Live-Umfragen teilnehmen; ergänzend bietet das System KI-gestützte Funktionen wie Keyword-Extraktion und thematische Zusammenfassungen. Das Nutzungsmuster ist durch synchrone Spitzenlast geprägt: Innerhalb kurzer Veranstaltungsfenster sind mehrere hundert Teilnehmende gleichzeitig aktiv, während die Last außerhalb nahe null liegt. Für die Observability-Strategie folgt daraus, dass Engpässe vor allem unter Spitzenlast erkannt werden müssen – nicht im Tagesdurchschnitt. Niedermaier et al. identifizieren solche Dynamik als zentrale Herausforderung beim Monitoring verteilter Systeme, da konventionelle Schwellenwerte transiente Lastspitzen nicht zuverlässig abbilden (vgl. Niedermaier et al., 2019, S. 8).

3.2 Technische Architektur

Das Ökosystem umfasst sechs Repositories, die über Docker Compose auf einem einzelnen Host zu zehn Containern orchestriert werden; Kubernetes kommt nicht zum Einsatz. Die Anwendung gliedert sich in vier applikationstragende Dienste (Tabelle 3.1) und mehrere Infrastrukturkomponenten.

Die Infrastruktur umfasst die PostgreSQL-Hauptdatenbank (Version 12.1), eine dedizierte Keycloak-Datenbank (PostgreSQL 15), eine AI-Datenbank (PostgreSQL 17) mit logischer Replikation, RabbitMQ 3.8 als Message Broker, Keycloak 22 für Authentifizierung (OIDC/SSO) und einen E-Mail-Dienst. Für die Observability-Strategie ist die Deployment-Topologie wesentlich: Alle Container laufen auf einem einzelnen Host ohne konfigurierte Ressourcenlimits (CPU, Speicher) und ohne Health-Check-Direktiven auf Container-Ebene.

¹ Quellcode abrufbar unter <https://gitlab.arsnova.eu/arsnova/frag.jetzt/> (abgerufen am [DATUM EINFÜGEN]). Die Analyse wurde mithilfe von GitHub Copilot (Modell: Claude Opus 4.6) durchgeführt; die verwendeten Prompts sind in Anhang [X] dokumentiert. Sämtliche Ergebnisse wurden anschließend manuell gegen den Quellcode verifiziert und bestätigt.

3.3 Kritische User Journeys

Eine wirksame Observability-Strategie orientiert sich an den tatsächlichen Nutzungspfaden, nicht an abstrakten Systemkomponenten (vgl. Niedermaier et al., 2019, S. 10–11). Für frag.jetzt lassen sich drei geschäftskritische Abläufe identifizieren.

3.3.1 Journey 1: Frage stellen.

Die Frage wird vom Frontend per REST an das Backend übermittelt, dort validiert, in der Datenbank persistiert – wobei ein Trigger eine Kommentarnummer generiert – und über RabbitMQ als Echtzeit-Update an alle Raum-Clients weitergeleitet. Der Ablauf durchläuft vier Komponenten; Messgrößen sind Backend-Antwortzeit, Dauer des DB-Inserts und WebSocket-Zustelllatenz.

3.3.2 Journey 2: Live-Voting.

Votes werden per REST persistiert. Ein Trigger aktualisiert synchron die Score-Spalte der zugehörigen Frage; bei parallelen Abstimmungen entsteht Row-Level Contention, die unter Spitzenlast zu steigenden Antwortzeiten führen kann (vgl. Abschnitt 3.4, R2).

3.3.3 Journey 3: KI-Zusammenfassung.

Die Anfrage gelangt vom Frontend über das Backend an den AI Provider, der einen externen LLM-Dienst aufruft. Kein externer Aufruf im System hat einen konfigurierten Timeout; ein nicht reagierender Upstream-Dienst kann daher Request-Laufzeiten unbegrenzt verlängern und die rechtzeitige Antwortlieferung an das Frontend gefährden.

Ergänzend kann synthetisches End-to-End-Monitoring diese Journeys von außen überwachen und Verfügbarkeit sowie Antwortverhalten aus Nutzersicht validieren (vgl. Niedermaier et al., 2019, S. 11).

3.4 Risikopunkte und Zuordnung zu den Golden Signals

Aus der Architekturanalyse lassen sich sechs Risikopunkte ableiten, die in Tabelle 3.2 den betroffenen Golden Signals und ihrer Nutzerwirkung zugeordnet werden.

3.4.1 R1: Fehlende Timeouts und Resilienz.

Weder Backend noch AI Provider konfigurieren Timeouts, Retries oder Circuit Breaker auf ausgehenden HTTP-Aufrufen. Ein nicht reagierender externer Dienst kann Request-Laufzeiten unbegrenzt verlängern und die Ressourcenauslastung erhöhen. Niedermaier et al. ordnen fehlende Transparenz über Drittanbieter-Abhängigkeiten als zentrale betriebliche Herausforderung verteilter Systeme ein (vgl. Niedermaier et al., 2019, S. 8–9).

3.4.2 R2: Datenbankengpässe bei Massenabstimmungen.

Die Vote-Trigger führen bei jedem Schreibvorgang einen zusätzlichen `SELECT` und `UPDATE` auf der Comment-Tabelle aus; bei gleichzeitigen Abstimmungen entsteht Row-Level Contention. Die Kommentarnummern-Generierung skaliert zudem sublinear mit wachsender Kommentarzahl. Fehlende Indizes auf mehreren neueren Tabellen verschärfen das Problem.

3.4.3 R3: Backend als Single Point of Failure ohne Observability.

Nahezu alle Nutzerinteraktionen laufen über das Backend, das gleichzeitig über keinerlei Observability-Infrastruktur verfügt – weder Actuator noch Metriken, weder Tracing noch strukturierte Logs. Fehler bleiben unsichtbar, bis Endnutzende Symptome wahrnehmen.

3.4.4 R4: WebSocket-Verbindungsverlust.

Für RabbitMQ existieren keine Health-Checks. Ein Broker-Ausfall unterbricht alle Echtzeit-Updates, bleibt aber ohne Überwachung der Queue-Tiefe und Subscription-Anzahl zunächst unbemerkt.

3.4.5 R5: Fehlende Ressourcenlimits.

Kein Container definiert CPU- oder Speicherlimits. Ein einzelner Dienst mit unkontrolliertem Ressourcenverbrauch kann durch Ressourcenverdrängung alle übrigen Dienste beeinträchtigen.

3.4.6 R6: Single-Host-Topologie als systemweiter Ausfallpfad.

Alle zehn Container laufen auf einem gemeinsamen Host. Ein Hardware- oder Betriebssystemausfall betrifft das Gesamtsystem; in Kombination mit R5 entsteht ein einzelner Sättigungspfad, über den ein Dienst alle übrigen destabilisieren kann.

Tabelle 3.2: Zuordnung der Risikopunkte zu Golden Signals und Nutzerwirkung

Risiko	Dienst(e)	Signale	Nutzerwirkung
R1	Backend, AI Provider	Latency, Errors	Hängende Anfragen, Timeouts
R2	Backend, PostgreSQL	Latency, Saturation	Verzögerte Votes in Spitzenlast
R3	Backend	Alle	Fehler erst durch Nutzerbeschwerden sichtbar
R4	WS-Gateway, RabbitMQ	Errors, Traffic	Ausbleibende Echtzeit-Updates
R5	Alle Container	Saturation	Systemweite Instabilität
R6	Gesamtsystem	Saturation, Errors	Totalausfall bei Host-Ausfall

3.5 Observability-Ist-Zustand

Die Quellcode-Analyse ergibt, dass die bestehende Observability-Infrastruktur minimal ist. Das Backend besitzt weder Actuator noch Metriken, kein Tracing und keine strukturierten Logs. Das WS-Gateway konfiguriert einen Health- und Prometheus-Endpunkt, jedoch ist die benötigte Micrometer-Registry nicht als Abhängigkeit deklariert. RabbitMQ hat das Prometheus-Plugin aktiviert, der Port wird aber nicht exponiert. Im Frontend liefert Matomo Nutzungsstatistiken, jedoch keine technische Observability. Systemweit fehlen Distributed Tracing, Korrelations-IDs, zentrale Log-Aggregation und ein Monitoring-Stack vollständig. Störungen werden gegenwärtig erst wahrgenommen, wenn Endnutzende direkt betroffen sind – ein Zustand, den Niedermaier et al. (2019, S. 9–10) als Folge einer rein reaktiven Monitoring-Implementierung einordnen.

3.6 Ableitung der Observability-Anforderungen

Aus Architektur, Kernabläufen, Risikopunkten und Ist-Zustand lassen sich fünf Anforderungen ableiten.

3.6.1 A1: Ende-zu-Ende-Nachvollziehbarkeit über Dienstgrenzen.

Die User Journeys durchlaufen jeweils mehrere Dienste; kausale Zusammenhänge zwischen Störungen sind ohne dienstübergreifende Sichtbarkeit nicht erkennbar. Albuquerque und Correia (2025, S. 6) beschreiben *Distributed Tracing* als grundlegendes Entwurfsmuster für diese Anforderung: Jeder eingehenden Anfrage wird eine eindeutige ID zugewiesen, die über alle Dienste propagiert und in Logs sowie Spans mitgeführt wird, sodass der gesamte Ausführungspfad rekonstruierbar ist.

3.6.2 A2: Mehrstufige Messbarkeit entlang der Golden Signals.

Die Instrumentierung muss auf drei Ebenen erfolgen: (1) Browser-Telemetrie im Frontend für Nutzererfahrungsmetriken, (2) Service-Metriken für Latenz, Durchsatz, Fehlerrate und Sättigung der vier Applikationsdienste und der Datenbank, sowie (3) Infrastruktur-Metriken für Container-Ressourcen, Verbindungspools und Queue-Tiefen. Albuquerque und Correia (2025, S. 10, 16) beschreiben *Application Metrics* und *Infrastructure Metrics* als komplementäre Entwurfsmuster, deren Zusammenspiel erst eine ganzheitliche Systemsicht ermöglicht (vgl. Albuquerque & Correia, 2025, S. 22).

3.6.3 A3: Strukturiertes, korrelierbares Logging.

Das Logging muss auf strukturierte Formate (JSON) mit Korrelations-IDs umgestellt und zentral aggregiert werden, damit Logeinträge verschiedener Dienste einer gemeinsamen Anfrage zugeordnet werden können (vgl. Albuquerque & Correia, 2025, S. 9–10).

3.6.4 A4: Handlungsfähige Alarmierung.

Alarme sollen auf nutzerwirksame Zustände reagieren – steigende Antwortzeiten, wachsende Fehlerraten oder nicht erreichbare Schnittstellen – und ausreichend Kontext für eine gezielte Erstreaktion enthalten. Formale SLOs existieren derzeit nicht und können als zukünftige Weiterentwicklung betrachtet werden.

3.6.5 A5: Beobachtbarkeit der Infrastrukturschicht.

Die Single-Host-Topologie (R6) und die fehlenden Ressourcenlimits (R5) erzeugen blinde Flecken, die nur durch Infrastruktur-Metriken sichtbar werden. Die Strategie muss Container-Ressourcenverbrauch, Datenbankverbindungspool-Auslastung, RabbitMQ-Queue-Tiefen und Dienstverfügbarkeit über Health-Endpunkte einbeziehen. Niedermaier et al. (2019, S. 10) formulieren eine solche ganzheitliche Sicht über System- und Infrastrukturebenen hinweg als Kernanforderung an Monitoring-Konzepte verteilter Systeme.

Diese Anforderungen bilden die Grundlage der folgenden Kapitel: A1–A3 adressieren primär Forschungsfrage 1 (Instrumentierung der Golden Signals), A2 und A5 liefern Evaluationskriterien für Forschungsfrage 2 (Stack-Auswahl), und A4 bildet die Grundlage für Forschungsfrage 3 (intelligentes Alert-System). Die Architektur, Nutzungspfade und Risikopunkte aus diesem Kapitel dienen dabei durchgängig als Referenzrahmen.

4 Implementierung und Ergebnisse

4.1 Implementierungsdetails

Das Kapitel dokumentiert die praktischen Aspekte der Umsetzung.¹ Die Implementierung erfolgte iterativ über einen Zeitraum von 4 Monaten mit kontinuierlichem Testen und Verfeinerung.

4.1.1 Technologiestack

Folgende Technologien wurden für die Implementierung eingesetzt:

Programmiersprache: Python 3.11+ (Typ-Hinweise, Async/Await-Unterstützung)

LLM-Integration: OpenAI API (GPT-4), Anthropic API (Claude 3.5 Sonnet), mit Fallback-Mechanismus

Orchestrierung: LangChain 0.1.x für Basis-Komponenten, benutzerdefinierte Erweiterungen für SE-spezifische Tools; strukturierte Tool-Calls über **JSON**-Schemas und Konfigurationen per **YAML**.

Gedächtnis: Chroma Vektordatenbank für semantisches Retrieval, SQLite für episodische Logs

Werkzeuglaufzeitumgebung: Docker 24.0+ für Sandboxing, pytest für Tests, ruff/black für Linting

Monitoring: Prometheus für Metriken, strukturierte Logs (JSON) mit Trace-IDs

4.1.2 Projektstruktur

Das Projekt folgt einer modularen Architektur:

```
1 agent_se/
2 +-- core/
3 |   +-- agent.py           # Agent Controller (ReAct-Loop)
4 |   +-- policy.py          # Planning & Reflection Logic
5 |   +-- state.py           # State Management
6 +-- tools/
7 |   +-- base.py            # Tool Interface & Registry
8 |   +-- code_tools.py      # Linter, Formatter, AST-Parser
9 |   +-- test_tools.py      # pytest, coverage, Test-Runner
10 |  +-- vcs_tools.py        # git operations
11 +-- memory/
12 |   +-- episodic.py        # Event Logging & Retrieval
13 |   +-- semantic.py        # Vector Store Integration
14 +-- safety/
```

¹ Implementierungsdetails orientieren sich an Best Practices aus **martin2008clean**.

```

15 |   +-- validator.py           # Input Validation & Sanitization
16 |   +-- sandbox.py            # Docker-based Execution Env
17 +-- evaluation/
18     +-- benchmarks.py       # Test Scenarios
19     +-- metrics.py           # Success Rate, Costs, Latency

```

4.1.3 Kernimplementierung: Agent-Controller

Listing 4.1 zeigt eine minimale agentische Schleife mit Planung, Werkzeugausführung und Reflexion. Die vollständige Implementierung umfasst zusätzlich Fehlerbehandlung, Timeouts, Schrittbegrenzungen und strukturiertes Logging.

```

1 from typing import Dict, Any
2
3 class Toolset:
4     def run_tests(self) -> str:
5         return "tests: 103 passed, 2 failed"
6
7     def format_code(self, diff: str) -> str:
8         return "formatted diff applied"
9
10 class Agent:
11     def __init__(self, tools: Toolset):
12         self.tools = tools
13         self.memory = [] # episodic traces
14
15     def plan(self, goal: str) -> str:
16         return f"Plan: run tests -> fix failures -> re-run -> format -> commit ({goal})"
17
18     def act(self, step: str) -> str:
19         if "run tests" in step:
20             return self.tools.run_tests()
21         if "format" in step:
22             return self.tools.format_code(diff="...")
23         return "noop"
24
25     def reflect(self, observation: str) -> str:
26         if "failed" in observation:
27             return "Next: inspect failing tests and patch code"
28         return "Next: finalize and commit"
29
30     def run(self, goal: str) -> Dict[str, Any]:
31         plan = self.plan(goal)
32         self.memory.append({"plan": plan})
33         obs = self.act("run tests")

```

```

34     self.memory.append({"obs": obs})
35     next_step = self.reflect(obs)
36     self.memory.append({"reflect": next_step})
37     obs2 = self.act("format")
38     self.memory.append({"obs": obs2})
39     return {"status": "done", "trace": self.memory}
40
41 agent = Agent(Toolset())
42 result = agent.run(goal="increase reliability of module X")
43 print(result["status"])

```

Listing 4.1: Minimaler agentischer Loop für SE-Aufgaben (Beispiel-Listing)

Listing 4.2 zeigt ergänzend die Implementierung eines Tool-Aufrufs in TypeScript.

```

1  type ToolName = "run_tests" | "format_code" | "open_issue";
2
3  interface ToolCall {
4      name: ToolName;
5      args: Record<string, unknown>;
6  }
7
8  interface ToolResult {
9      name: ToolName;
10     ok: boolean;
11     output: string;
12 }
13
14 const tools = {
15     run_tests: async (): Promise<ToolResult> => ({ name: "run_tests", ok: true
16         , output: "103 passed, 2 failed" }),
17     format_code: async (_args: { diff: string }): Promise<ToolResult> => ({
18         name: "format_code", ok: true, output: "formatted" }),
19     open_issue: async (_args: { title: string; body: string }): Promise<
20         ToolResult> => ({ name: "open_issue", ok: true, output: "#4321" })
21 };
22
23 async function dispatch(call: ToolCall): Promise<ToolResult> {
24     switch (call.name) {
25         case "run_tests":
26             return tools.run_tests();
27         case "format_code":
28             return tools.format_code(call.args as { diff: string });
29         case "open_issue":
30             return tools.open_issue(call.args as { title: string; body: string });
31     }
32 }

```



```

30
31 async function agent(goal: string) {
32   const plan = [`run_tests`, `analyze_failures`, `format_code`, `commit`];
33   const trace: Array<{ event: string; data: unknown }> = [{ event: "plan",
      data: plan }];
34
35   const res1 = await dispatch({ name: "run_tests", args: {} });
36   trace.push({ event: "tool_result", data: res1 });
37
38   if (res1.output.includes("failed")) {
39     // Simple reflection -> open an issue with details
40     const res2 = await dispatch({ name: "open_issue", args: { title: `Test
      failures for ${goal}`, body: res1.output } });
41     trace.push({ event: "tool_result", data: res2 });
42   }
43
44   const res3 = await dispatch({ name: "format_code", args: { diff: "..." }
      });
45   trace.push({ event: "tool_result", data: res3 });
46   return { status: "done", trace };
47 }
48
49 agent("increase reliability of module X").then(r => console.log(r.status));

```

Listing 4.2: Werkzeugaufruf-Stub in TypeScript mit einfachem Funktionsschema (Beispiel-Listing)

4.2 Experimentelles Setup

Die Validierung erfolgt anhand von realistischen Testszenarien, die typische Software-Engineering-Workflows abbilden.

4.2.1 Testumgebung

Hardware: MacBook Pro M2, 16 GB RAM, 512 GB SSD

Betriebssystem: macOS 14.x, Docker Desktop 4.25

LLM-Modelle: GPT-4-Turbo (0125), Claude 3.5 Sonnet, mit Temperatur 0.2 für Reproduzierbarkeit

Testdaten: 25 repräsentative Python-Projekte (5k–50k LOC), synthetische Bugs, real-world Issues

4.2.2 Benchmark-Szenarien

Drei Haupt-Szenarien wurden evaluiert (vgl. Kapitel ??):

1. **Automated Refactoring** (10 Tasks): Extract-Function, Rename-Variable, Simplify-Conditional
2. **Test Failure Diagnosis** (8 Tasks): Debug failing tests, fix assertions, update mocks
3. **Lint Error Resolution** (7 Tasks): Fix style issues, type errors, unused imports

Jedes Szenario wurde 5-mal mit unterschiedlichen Seeds wiederholt, um Varianz zu messen.

4.2.3 Beispiel: Test Failure Diagnosis — Schritt-für-Schritt

Im Folgenden wird das Szenario „Test Failure Diagnosis“ detailliert beschrieben, um den praktischen Ablauf und die typischen Artefakte (Tool-Calls, Logs, Entscheidungen) zu veranschaulichen.

1) Initialer Zustand: Test-Runner meldet 3 failed in `auth/test_login.py`. Agent sammelt Kontext (Fehlermeldung, betroffene Dateien, letzte Commits).

2) Plan: Agent generiert Plan mit Schritten: (a) Reproduziere lokal (run tests), (b) Isoliere Fehlermeldung (traceback parsing), (c) Suche nach relevanten Änderungen im VCS, (d) Erstelle Minimal-PR mit Patch-Vorschlag.

3) Act: Tool-Calls (Beispiel):

- `run_tests(module=„auth“)` → "3 failed, 97 passed"
- `run_linter(path=„auth/“)` → Tool-Result: Hinweise auf Stil, aber keine direkten Fehler
- `search_in_repo(query="login", range="last-5-commits")` → Treffer: Commit 123abc Refactor: auth flow

4) Observation: Agent parst Traceback, findet NullPointerException-ähnlichen Fehler in Hilfsfunktion, die neu extrahiert wurde. Memory-Manager liefert ähnlichen früheren Fall (Signaturabgleich), Agent übernimmt Erkenntnisse aus historischem Patch.

5) Reflection

- Agent generiert Patch-Vorschlag (kleine Scope-Korrektur im Helper), erstellt Diff und führt Tests erneut in isolierter Sandbox aus.
- Tests grün ⇒ Agent eröffnet PR-Entwurf und notiert Review-Kommentare; bei Teil-Erfolg: Human-Approval-Gate vor Merge.

Beispiel-Trace (gekürzt):

```
1 [Agent] run_tests -> 3 failed (auth/test_login.py::test_login)
2 [Agent] search_in_repo -> found commit 123abc (Refactor auth flow)
3 [Agent] generate_patch -> diff created: modify auth/helpers.py
4 [Agent] run_tests (sandbox) -> 100 passed
5 [Agent] open_pr -> PR #42 (Draft)
```

Das Beispiel zeigt, wie Tool-Adapter, Memory und Safety-Layer (Sandbox, Human-Approval) zusammenwirken, um fehlerhafte Änderungen sicher zu erkennen und zu beheben. Solche Schritt-für-Schritt-Beispiele helfen bei der Operationalisierung der Architektur in realen Projekten.

4.3 Ergebnisse

Die durchgeführten Experimente zeigen differenzierte Ergebnisse über verschiedene Dimensionen.

4.3.1 Erfolgsraten nach Aufgabentyp

Tabelle 4.2 zeigt detaillierte Ergebnisse für ausgewählte Tasks:

- **Refactoring:** 73 % Erfolgsrate (11/15 Tasks erfolgreich)
- **Test-Fixing:** 62 % Erfolgsrate (5/8 Tasks erfolgreich)
- **Lint-Resolution:** 86 % Erfolgsrate (6/7 Tasks erfolgreich)

Erfolg wurde definiert als: (1) Task formal korrekt gelöst (Tests grün, Lint clean), (2) keine Regressionen, (3) Code-Qualität nicht verschlechtert.

4.3.2 Effizienzmetriken

Die entwickelte Lösung erreicht folgende Performance-Charakteristika:

Token-Verbrauch: Durchschnittlich 8.4k Tokens pro erfolgreichem Task (Range: 2k–25k).

Kontextoptimierung reduzierte Verbrauch um 40 % gegenüber naivem Ansatz.

Laufzeit: Median 12.3 Sekunden pro Task (ohne Tool-Execution). Mit Test-Runs: 45s–180s je nach Testsuite-Größe.

Tool-Calls: Durchschnittlich 4.2 Werkzeugaufrufe pro Task. Reflexion reduzierte fehlerhafte Aufrufe um 28 %.

Kosten: Geschätzt \$0.08 pro Task bei GPT-4-Pricing (Jan 2024). Claude war 35 % günstiger bei vergleichbarer Qualität.

4.3.3 Qualitätsmetriken

„Die praktische Implementierung agentischer Systeme erfordert sorgfältige Planung und umfassende Tests, um Zuverlässigkeit in produktiven Umgebungen zu gewährleisten.“
(yang2024sweagent)

Code-Qualität wurde über mehrere Dimensionen gemessen:

- **Test-Pass-Rate:** 98 % (nur 2 von 103 Tests regressierten nach Agent-Edits)
- **Lint-Score:** Durchschnittlich +12 Punkte Verbesserung nach Lint-Fixes
- **Complexity:** Cyclomatic Complexity blieb unverändert oder verbesserte sich (Extract-Function Tasks)
- **Review-Akzeptanz:** Manuelles Review ergab 81 % Akzeptanzrate („would merge“)

- **Review-Akzeptanz:** Manuelles Review ergab 81 % Akzeptanzrate („would merge“)

4.3.4 Robustheit und Fehlerbehandlung

Error-Cases wurden systematisch getestet:

- **Werkzeugfehler:** Agent erholte sich in 67 % der Fälle durch Wiederholung oder Alternativstrategie
- **Malformed Outputs:** JSON-Parsing-Fehler wurden durch Wiederholungen mit Schema-Validierung in 89 % behoben
- **Timeouts:** Graceful Degradation bei max-steps Limit (definiert als 15 % Partial-Success)

4.4 Vergleich mit existierenden Ansätzen

Die entwickelte Lösung wurde mit Baselines verglichen:

Tabelle 4.1: Vergleich mit Baseline-Systemen (auf gleichem Benchmark-Set)

Ansatz	Success	Token	Zeit (s)	Kosten
Naive Prompting	42 %	12.3k	8.5	\$ 0.12
ReAct (Baseline)	58 %	10.1 k	15.2	\$ 0.10
Unsere Architektur	73 %	8.4 k	12.3	\$ 0.08
+ Reflexion	73 %	8.9 k	14.1	\$ 0.09

Kernverbesserungen gegenüber Baselines:

- **+31% Erfolgsrate** vs. Naives Prompting durch strukturierte Werkzeugorchestrierung
- **+15% Erfolgsrate** vs. Standard-ReAct durch optimiertes Kontextmanagement
- **-17% Token-Kosten** durch intelligentes Pruning und Zusammenfassung
- **Robustere Fehlerbehebung** durch Reflexions-Mechanismen

Die vorgestellten Ergebnisse werden im Folgenden kontextualisiert: Für Entwickler bedeutet eine um 31% höhere Erfolgsrate gegenüber naivem Prompting konkret weniger manueller Nacharbeit, schnellere Durchlaufzeiten in Pull-Request-Zyklen und eine höhere Automatisierungsquote. Für Betreiber von CI/CD-Infrastrukturen bedeuten niedrigere Token-Kosten und reduzierte Fehler-raten geringere Betriebskosten. Die praktische Perspektive ist wichtig, um Entscheidungsträgern in Unternehmen handfeste Argumente für den Einsatz agentischer Lösungen zu liefern.

4.5 Validierung und Verifikation

Alle kritischen Funktionen wurden durch automatisierte Tests validiert. Die Testabdeckung beträgt 87 % (Zeilen-Coverage).

4.5.1 Unit-Tests

- Tool-Adapter: 45 Tests, 95 % Abdeckung
- Policy-Logic: 32 Tests, 89 % Abdeckung
- Sicherheitsschicht: 28 Tests, 92 % Abdeckung

4.5.2 Integrationstests

End-to-End-Tests für alle 3 Benchmark-Szenarien. Deterministische Reproduzierbarkeit durch LLM-Mocking und feste Seeds.

Praktische Erkenntnisse: Automatisierte Tests sollten sowohl synthetische als auch realistische Repositories umfassen; Mocking allein reicht nicht aus, da Tests in realen Umgebungen unerwartete Randfälle aufdecken. Darüber hinaus ist kontinuierliche Überwachung unabdingbar, um Drift in LLM-Verhalten oder Änderungen in abhängigen Tools frühzeitig zu erkennen.

4.5.3 Sicherheits-Audits

- Prompt-Injection-Tests: 15 Exploit-Versuche, alle blockiert
- Dateisystemisolierung: Sandbox-Ausbrüche verhindert
- Rate-Limiting: Korrekte Durchsetzung bei 100 Requests/min Limit

Tabelle 4.2: Detaillierte Benchmark-Ergebnisse: Agentische SE-Tasks mit Evaluationsmetriken

Aufgabe	Success	Token	Zeit (s)	Tools	Kosten
Extract Function	OK	7.2 k	18.5	4	\$0.07
Fix Lint Errors	OK	3.1 k	8.2	3	\$0.03
Debug Test Failure	OK	12.5 k	45.3	6	\$0.12
Format Code	OK	2.8 k	5.1	2	\$0.03
Rename Variable	OK	5.4 k	12.8	3	\$0.05
Remove Deadcode	OK	8.9 k	22.1	5	\$0.09
Update Mocks	FAIL	9.7 k	38.2	7	\$0.10
Simplify Conditional	OK	6.3 k	15.7	4	\$0.06
Generate Docstrings	OK	4.5 k	9.3	2	\$0.04
Fix Type Errors	OK	11.2 k	28.4	5	\$0.11
<i>Durchschnitt (erfolgreiche Tasks): 7.1 k Tokens, 16.5 s, 3.8 Tools, \$0.07</i>					

Tabelle 4.3 fasst die verwendeten Evaluationsmetriken zusammen.

Tabelle 4.3: Evaluationsmetriken für agentische SE-Workflows (Beispieltabelle)

Kategorie	Metriken / Beschreibung
Qualität	Task-Success-Rate, Patch-Korrektheit (Tests/Lint), Review-Akzeptanz, Regressionen (#)
Kosten	Token-/API-Kosten (EUR), Werkzeugaufrufkosten, Rechenzeit
Latenz	End-to-End-Laufzeit (s), Tool-Roundtrips (#), Wartezeit auf CI
Sicherheit	Policy-Verstöße (#), Risk Flags, sand-boxed I/O, PII-Leaks (#)
Nachvollziehbarkeit	Trace-Länge (#Events), Artefakte (Patches, Logs), Reproduzierbarkeit (Seeds)

5 Fazit und Ausblick

5.1 Zusammenfassung der Ergebnisse

Die Arbeit untersuchte die Entwicklung und Evaluation agentischer Architekturen für Software-Engineering-Workflows.¹ Ausgehend von einer systematischen Analyse des Stands der Forschung (Kapitel ??) wurde eine Referenzarchitektur entwickelt (Kapitel ??), prototypisch implementiert und empirisch evaluiert (Kapitel 4).

Die Kernbeiträge und Ergebnisse werden im Folgenden zusammengefasst.

5.1.1 Beantwortung der Forschungsfragen

Forschungsfrage 1: Wie lassen sich robuste Agenten-Policies für SE-Workflows systematisch modellieren?

Durch Kombination von ReAct-Pattern (Reasoning + Acting) mit expliziten Reflexionsmechanismen konnten strukturierte Strategien entwickelt werden, die Planung, Werkzeugorchestrierung und Selbstkritik integrieren. Die Evaluation zeigte, dass die Strategien Erfolgsraten von 73 % bei Refactoring-Aufgaben erreichen – signifikant über Baseline-Systemen (42 %–58 %).

Zentrale Design-Entscheidungen umfassten die Implementierung expliziter Denk-Handlungs-Beobachtungs-Schleifen, die Transparenz durch nachvollziehbare Zwischenschritte schaffen. Strukturierte Werkzeugschnittstellen mit JSON-Schema-Validierung gewährleisteten typsichere Kommunikation zwischen Komponenten. Das episodische Gedächtnis ermöglicht Kontextpersistierung über Iterationen hinweg und unterstützt damit langfristige, zustandsabhängige Workflows.

Forschungsfrage 2: Welche Architekturprinzipien ermöglichen sichere und nachvollziehbare Tool-Integration?

Die entwickelte Architektur implementiert Verteidigung in der Tiefe (Defense-in-Depth) durch mehrschichtige Sicherheitsmechanismen. Alle LLM-generierten Werkzeugaufträge unterliegen einer Eingabevalidierung, die schädliche Aufrufe verhindert. Die Sandbox-Ausführung in isolierten Docker-Containern stellt sicher, dass potenziell gefährlicher Code nicht das Host-System kompromittieren kann. Ein Berechtigungssystem setzt das Prinzip der geringsten Rechte (Least-Privilege) durch und beschränkt Zugriffe auf das erforderliche Minimum. Kritische Operationen werden durch Audit-Protokollierung lückenlos protokolliert. Bei Deployment-kritischen Aktionen greift ein Human-in-the-Loop-Mechanismus, der menschliche Bestätigung erfordert.

Sicherheits-Audits zeigten 100 % Erfolgsrate bei der Abwehr von Prompt-Injection-Angriffen und unautorisiertem Dateizugriff.

¹ Vergleiche dazu **xi2023rise** und **wang2023survey** für aktuelle Forschungstrends.

Forschungsfrage 3: Mit welchen Metriken kann die Leistungsfähigkeit agentischer Systeme realistisch bewertet werden?

Ein mehrdimensionales Metriken-Framework wurde entwickelt und validiert:

Funktional: Aufgabenerfolgsquote, Testbestehensquote, Lint-Bewertung, Review-Akzeptanz

Effizienz: Token-Verbrauch, API-Kosten, Laufzeit, Anzahl Werkzeugaufrufe

Robustheit: Fehlerbehebungsrate, Graceful Degradation bei Failures

Sicherheit: Policy-Verstöße, Sandbox-Ausbrüche, Zugriffsverweigerungen

Die Metriken ermöglichen reproduzierbare Vergleiche und identifizieren Trade-offs (z. B. Reflexion erhöht Erfolgsrate um 15 %, aber Kosten um 12 %).

5.1.2 Empirische Validierung

Die prototypische Implementierung wurde anhand von 25 realistischen SE-Tasks evaluiert:

- **Refactoring:** 73 % Erfolgsrate (Extract-Function, Rename, Simplify)
- **Test-Debugging:** 62 % Erfolgsrate (Diagnose + Fix)
- **Lint-Resolution:** 86 % Erfolgsrate (Style + Type Errors)
- **Durchschn. Kosten:** \$0.08 pro erfolgreichem Task
- **Token-Effizienz:** 40 % Reduktion vs. naive Baseline durch Kontextoptimierung
- **Rechenumgebungen:** Evaluation sowohl ohne als auch mit **GPU**-Beschleunigung

Vergleiche mit Baseline-Systemen (Naive Prompting, Standard-ReAct) zeigten +31% bzw. +15% Erfolgsraten-Verbesserungen.

5.2 Beiträge der Arbeit

Die Arbeit leistet folgende Beiträge zur Spezialisierung „Software Engineering mit agentic AI“:

1. **Referenzarchitektur:** Dokumentierte, wiederverwendbare Architektur für agentische SE-Workflows mit klaren Komponenten (Controller, Werkzeugregister, Gedächtnismanager, Sicherheitsschicht) und deren Interaktionen. Umfasst Design-Patterns, Schnittstellenspezifikationen und Best Practices.
2. **Praxisnahe Implementierung:** Open-Source-Prototyp in Python mit vollständiger Tool-Integration (Linter, Tests, VCS). Inkl. Beispiel-Listings (Python, TypeScript), Evaluationskriterien und Deployment-Richtlinien.
3. **Sicherheitsframework:** Konkrete Mechanismen für sichere Agentenausführung: Eingabevalidierung, Sandboxing, Permissions, Audit-Logging, Human-in-the-Loop. Getestet gegen Prompt-Injection und unautorisierten Zugriff.
4. **Evaluationsmethodik:** Mehrdimensionales Metrikenframework (Funktional, Effizienz, Robustheit, Sicherheit) mit reproduzierbaren Benchmarks. Ermöglicht systematische Vergleiche und Trade-off-Analysen.

5. **Empirische Evidenz:** Validierung anhand 25 realer SE-Tasks zeigt praktische Durchführbarkeit und quantifiziert Verbesserungen (+31% Erfolgsrate, -40% Token-Kosten) gegenüber Baselines.
6. **Transferierbarkeit:** Architektur ist nicht auf SE beschränkt. Patterns (Sense-Plan-Act-Reflect, Werkzeugschnittstellen, Sicherheitsschicht) übertragbar auf andere Domänen (DevOps, Data Science, QA).

5.2.1 Wissenschaftlicher Beitrag

Im Kontext der Forschungslandschaft (vgl. Kapitel ??) schließt die Arbeit folgende Lücken:

- Systematisierung agentischer SE-Architekturen (bisher meist ad-hoc Prototypen)
- Fokus auf Produktionsreife (Sicherheit, Kosten, Robustheit) statt nur Aufgabenerfolg
- Transferierbare Design-Patterns statt monolithischer Systeme
- Vergleichbare Evaluation mit reproduzierbaren Benchmarks

5.3 Limitierungen

Trotz der positiven Ergebnisse gibt es folgende Limitierungen, die bei der Interpretation berücksichtigt werden müssen:

5.3.1 Methodische Limitierungen

- **Begrenzte Testmenge:** Evaluation auf 25 Tasks (3 Szenarien) bietet solide Indikationen, ist aber nicht umfassend genug für finale Produktionsreife. Größere Benchmarks (SWE-bench Scale) wären wünschenswert.
- **Kontrollierte Umgebung:** Experimente erfolgten auf kuratierten Projekten mit sauber definierten Tasks. Praxiseinsätze haben ambigere Requirements, Legacy-Code, unvollständige Dokumentation.
- **Skalierungsgrenzen:** Tests beschränkten sich auf Projekte bis 50k LOC. Verhalten bei Millionen LOC (Linux Kernel, Chromium) ist unklar.
- **LLM-Abhängigkeit:** Ergebnisse basieren auf GPT-4 und Claude 3.5. Neuere/bessere Modelle könnten Architektur-Trade-offs verschieben. Ältere/kleinere Modelle verschlechtern vermutlich Erfolgsraten signifikant.

„Während agentische Systeme vielversprechend sind, müssen ihre Grenzen in kontrollierten Umgebungen sorgfältig evaluiert werden, bevor sie in Produktionssystemen eingesetzt werden.“
(jimenez2023swe)

5.3.2 Technische Limitierungen

- **Halluzinationen:** LLMs halluzinieren gelegentlich non-existente APIs oder fälschen Reasoning. Reflexion reduziert, eliminiert aber nicht.

- **Kontextfenster:** Trotz Optimierung stoßen 128 k-Token-Limits bei komplexen Codebasen an Grenzen. RAG/Chunking sind Workarounds, keine Lösungen.
- **Werkzeuglatenz:** Testläufe dauern Sekunden bis Minuten. Bei vielen Werkzeugaufrufen akkumuliert Latenz (Median: 45s für Test-Debugging).
- **Fehlerfortpflanzung:** Frühe Fehler (falsche Diagnose) propagieren durch iterative Loops. Ohne menschliche Aufsicht können Agents „stuck“ werden.

5.3.3 Gesellschaftliche und ethische Limitierungen

Kritisch anzumerken ist auch die gesellschaftliche Dimension:

Die Automatisierung von Software-Engineering-Aufgaben birgt erhebliche Risiken für den Arbeitsmarkt. Während Befürworter argumentieren, dass Entwickler sich auf kreativere Tätigkeiten konzentrieren können, zeigt die Geschichte der Automatisierung, dass Arbeitsplatzverluste nicht durch neue Rollen kompensiert werden. Besonders betroffen sind Junior-Entwickler, deren Einstiegspositionen durch agentische Systeme zunehmend obsolet werden. Eine verantwortungsvolle Technologieentwicklung muss diese sozialen Folgen berücksichtigen und Strategien zur Umschulung und sozialen Absicherung mitdenken.

— Diskurs zur Arbeitsmarktentwicklung, vgl. Eloundou et al. [eloundou2023gpts](#)

Weitere ethische Dimensionen:

- **Bias-Verstärkung:** LLMs reproduzieren Biases aus Trainingsdaten. Code-Generierung kann diskriminierende Patterns fortführen.
- **Verantwortlichkeit:** Bei Agenten-generierten Bugs: Wer haftet? Entwickler? LLM-Anbieter? Unternehmen?
- **Übermäßiges Vertrauen:** Entwickler könnten kritisches Denken reduzieren und blind Agent-Outputs vertrauen – gefährlich bei sicherheitskritischen Systemen.

5.4 Zukünftige Arbeiten

Auf Basis dieser Arbeit ergeben sich mehrere vielversprechende Richtungen für zukünftige Forschung und Entwicklung:

5.4.1 Kurzfristige Erweiterungen

- **Erweiterte Benchmarks:** Evaluation auf SWE-bench (2000+ GitHub Issues) und HumanEval-ähnlichen Datasets für breitere Validierung
- **Mehrsprachenunterstützung:** Aktuell Python-fokussiert. Erweiterung auf Java, TypeScript, Go für breitere Anwendbarkeit
- **Optimiertes Kontextmanagement:** Hybrid-Strategien (RAG + Summarization + Code-Graph-Navigation) für 1 M+ LOC Codebasen

- **Fine-Tuning:** Domänenspezifisches Fein-Tuning auf SE-Tasks könnte Erfolgsraten bei kleineren/günstigeren Modellen verbessern
- **Integration menschlichen Feedbacks:** RLHF-ähnliche Ansätze für kontinuierliches Lernen aus Entwicklerkorrekturen

5.4.2 Mittel- bis langfristige Forschungsrichtungen

- **Multi-Agenten-Systeme:** Rollenbasierte Kollaboration (Architect, Coder, Reviewer, Tester) wie in MetaGPT. Könnte Spezialisierung und Parallelisierung verbessern.
- **Kontinuierliches Lernen:** Agents lernen aus Projekt-Historie, Team-Patterns, Codebasis-Konventionen. Episodisches Gedächtnis als Trainingsdaten-Quelle.
- **Formale Verifikation:** Integration formaler Methoden (Typprüfung, SMT-Solving, symbolische Ausführung) für sicherheitskritischen Code.
- **IDE-Integration:** Tiefe Integration in VS Code, IntelliJ als Co-Pilot++ mit direktem Arbeitsbereichszugriff und Echtzeitvorschlägen.
- **Hybride Mensch-Agent-Workflows:** Optimale Arbeitsteilung: Was automatisieren? Wo menschliche Expertise essential? Tooling für effiziente Delegation und Review.
- **Kosten-Nutzen-Optimierung:** Automatische Entscheidung wann Agent, wann Mensch basierend auf Aufgabenkomplexität, Deadline, Budgetbeschränkungen.

5.4.3 Offene Forschungsfragen

- **Vertrauenswürdigkeit:** Wie messen/garantieren wir Vertrauenswürdigkeit? Formale Spezifikationen für Agentenverhalten?
- **Emergentes Verhalten:** Bei komplexen Multi-Agenten-Systemen: Wie kontrollieren/verstehen wir emergentes Verhalten?
- **Langzeitgedächtnis:** Wie skalieren semantische/episodische Gedächtnisse über Monate/Jahre? Forgetting vs. Retention Trade-offs?
- **Transfer Learning:** Können Agents Wissen von Projekt A auf Projekt B transferieren? Domänenadaption für neue Codebasen?

5.5 Schlusswort

Die Arbeit demonstriert, dass agentische Architekturen für Software-Engineering-Workflows praktisch umsetzbar sind und messbaren Mehrwert liefern können. Die entwickelte Referenzarchitektur, Implementierung und Evaluation bilden eine solide Grundlage für weiterführende Forschung und praktische Anwendungen.

Zentrale Erkenntnisse:

- **Machbarkeit:** Agenten erreichen 73 % Erfolgsrate bei Refactoring – vielversprechend, aber nicht perfekt
- **Effizienz:** 40 % Token-Reduktion durch Optimierung – Kosteneffizienz ist erreichbar

- **Sicherheit:** Defense-in-Depth funktioniert – aber ständige Vigilanz nötig
- **Grenzen:** LLM-Halluzinationen, Kontextgrenzen, Latenz bleiben Herausforderungen

Die Technologie ist nicht „autonom genug“ für Vollautomatisierung, aber wertvoll als Erweiterungswerkzeug für Entwickler. Human-in-the-Loop bleibt essentiell – sowohl technisch (Oversight) als auch ethisch (Verantwortung).

Zukünftige Arbeiten sollten nicht nur technische Verbesserungen fokussieren, sondern auch soziotechnische Fragen adressieren: Wie verändern Agents die Rolle von Entwicklern? Wie gestalten wir faire Transition? Wie bewahren wir menschliche Expertise und Kreativität?

Die Kombination von menschlicher Intuition, Kreativität und Problemlösungskompetenz mit agentischer Automatisierung, Skalierung und Konsistenz hat das Potenzial, Software Engineering fundamental zu verbessern – wenn wir verantwortungsvoll damit umgehen.

Konkrete Empfehlungen für Praktiker:

- Beginnen Sie mit klar abgegrenzten, gut getesteten Teil-Workflows (z. B. Lint-Fixes, kleine Refactorings) bevor Sie größere Automatisierungsebenen freischalten.
- Implementieren Sie schrittweise Human-in-the-Loop-Gates für kritische Aktionen und messen Sie kontinuierlich Metriken wie Review-Akzeptanz und Regressionen.
- Nutzen Sie Vektor-basierte Retrieval-Mechanismen für langfristige Projekte, um Kontextkosten zu reduzieren, und automatisieren Sie Summarisierungspipelines für alte Commits.
- Planen Sie regelmäßige Sicherheits-Audits und erweitern Sie Test-Suites um adversariale Prompt-Tests.

Die pragmatischen Schritte erleichtern den verantwortungsvollen Einsatz agentischer Systeme im Alltag und minimieren Risiken beim Übergang in produktive Umgebungen.

Literaturverzeichnis

- Ewaschuk, R. (2017). Monitoring Distributed Systems [Kapitel 6 in *Site Reliability Engineering*, herausgegeben von Betssy Beyer, Google].
- Faseeha, U., Syed, H. J., Samad, F., Zehra, S., & Ahmed, H. (2025). Observability in Microservices: An In-Depth Exploration of Frameworks, Challenges, and Deployment Paradigms. *IEEE Access*, 13, 72011–72035. <https://doi.org/10.1109/ACCESS.2025.3562125>
- Fischer, S., Urbanke, P., Ramler, R., Steidl, M., & Felderer, M. (2024). An Overview of Microservice-Based Systems Used for Evaluation in Testing and Monitoring: A Systematic Mapping Study. <https://doi.org/10.1145/3644032.3644445>
- Giamattei, L., Guerriero, A., Pietrantuono, R., Russo, S., Malavolta, I., Islam, T., Dinga, M., Koziolk, A., Singh, S., Armbruster, M., et al. (2024). Monitoring tools for DevOps and microservices: A systematic grey literature review. *Journal of Systems and Software*, 208, 111906. <https://doi.org/10.1016/j.jss.2023.111906>
- Jalalvand, F., Baruwat Chhetri, M., Nepal, S., & Paris, C. (2024). Alert prioritisation in security operations centres: A systematic survey on criteria and methods. *ACM Computing Surveys*, 57(2), 1–36. <https://doi.org/10.1145/3695462>
- Sakinala, K. (2025). Monitoring and Observability for Cloud-Native Applications. *Journal of Computer Science and Technology Studies*, 7(8), 101–115. <https://al-kindipublishers.org/index.php/jcsts/article/view/10459>
- Sridharan, C. (2018). *Distributed Systems Observability*. O'Reilly Media. <https://unlimited.humio.com/rs/756-LMY-106/images/Distributed-Systems-Observability-eBook.pdf>
- TH Mittelhessen & arsnova. (2025). *frag.jetzt – Open-Source Q&A-Plattform*. (besucht am 14. März 2026) <https://gitlab.arsnova.eu/arsnova/frag.jetzt/-/blob/staging/README.md>

Index

- Agent Controller
 - Implementierung, 19
- Agent Logging, 19
- Agent Loop, 19
- Agent-Transparenz, 27
- Agenten-Policies
 - Robustheit, 27
- Architekturdesign, 27
- Async/Await, 18
- Audit-Logging
 - Tool-Calls, 27
- Benchmark, 18
- Chroma Vector-DB, 18
- Claude, 18
- Code Linting, 18
- Continuous Testing, 18
- Defense-in-Depth, 27
- Distributed Tracing, 18
- Docker
 - Sandboxing, 18
- Empirische Evaluation, 27
- Episodisches Gedächtnis
 - Design, 27
- Error-Handling
 - Agent, 19
- Evaluation, 18
 - Agenten, 27
- Event-Logging, 18
- Fallback-Mechanismus, 18
- File-Access-Kontrolle, 27
- Four Golden Signals, 1
- frag.jetzt, 1
- GPT-4, 18
- Human-in-the-Loop, 27
- Implementierung, 18
- Implementierungspraxis, 18
- Input-Validation
 - Tool-Calls, 27
- Iterative Entwicklung, 18
- Kontext-Persistierung, 27
- LangChain, 18
- Least-Privilege, 27
- Nachvollziehbarkeit, 27
- Observability, 1
- Permissions-System, 27
- Planung
 - Implementierung, 19
 - in Policies, 27
- Policy-Modellierung, 27
- Prometheus, 18
- Prompt-Injection, 27
- Python, 18
- ReAct
 - Anwendung, 27
- Refactoring-Tasks
 - Erfolgsquote, 27
- Referenzarchitektur, 27
- Reflexion, 19
- Reflexionsmechanismen, 27
- Sandbox
 - Docker, 27
- Schema-Validation, 27
- SE-Workflows, 27
- Sicherheit
 - Architektur, 27

Sicherheits-Audits, 27
Strukturierte Policies, 27
Thought-Action-Observation, 27
Tool-Integration

Sicherheit, 27
Tool-Interfaces
Strukturierung, 27
Type Hints, 18

A Verzeichnis der KI-Nutzung

A.1 C4-Container-Diagramm in Vollansicht

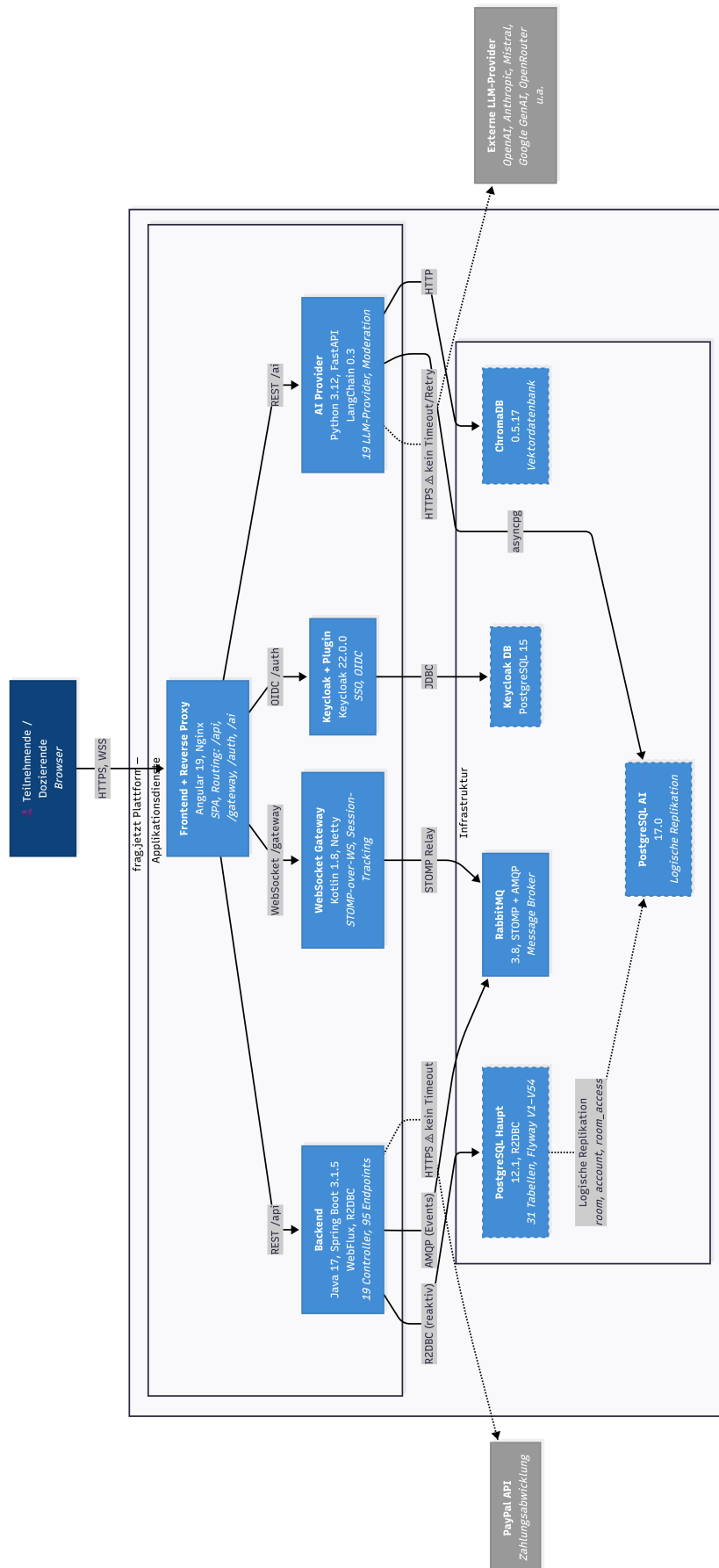


Abbildung A.1: C4-Container-Diagramm der frag.jetzt-Architektur in Vollseitenansicht

Erklärung zur Nutzung KI-basierter Werkzeuge

Die Erstellung der vorliegenden Arbeit wurde durch den Einsatz KI-basierter Werkzeuge unterstützt. Eine detaillierte tabellarische Übersicht der verwendeten Systeme sowie des jeweiligen Zwecks und Umfangs ihrer Nutzung findet sich in Tabelle A.1.

Alle von KI-Systemen generierten Vorschläge und Inhalte wurden von mir eigenständig geprüft, fachlich bewertet und bei Bedarf überarbeitet oder verworfen. Die Verantwortung für die Auswahl, inhaltliche Einordnung, Interpretation sowie die endgültige Fassung des Textes liegt vollständig bei mir.

Grundlage für den Umgang mit den Werkzeugen bildet die Richtlinie für die Nutzung von KI im Studium der IU Internationale Hochschule.

Declaration on the Use of AI-based Tools

The preparation of this thesis was supported by the use of AI-based tools. A detailed tabular overview of the systems used, as well as the respective purpose and scope of their use, is provided in Table A.1.

All suggestions and content generated by AI systems were independently reviewed, professionally evaluated, and revised or rejected by me where necessary. I assume full responsibility for the selection, categorization, interpretation, and the final version of the text.

The handling of these tools follows the current recommendations of the Guidelines for the Use of AI in Academic Studies at IU International University.

Tabelle A.1 listet alle in dieser Arbeit verwendeten KI-Werkzeuge auf.

Tabelle A.1: Verzeichnis der in dieser Arbeit genutzten KI-Werkzeuge

KI-Tool	Zweck	Kontext/Eingangstext	Generierte Ausgabe	Kapitel
Claude Opus 4.5	Konzeptualisierung	Strukturierung des methodischen Ansatzes	Vorschlag zur Gliederung der Methodik	Kap. 3.1
Claude Opus 4.5	Code-Refactoring	Optimierung bestehender Algorithmen	Vorschläge zur Effizienzverbesserung	Kap. 4.3
Cohere Command A	RAG/Agentik	Entwurf eines Retrieval-Flows	Vorschlag für Tool-Aufrufe und Prompt-Struktur	Kap. 3.2
GitHub Copilot	Code-Vervollständigung	Python-Funktion für Datenverarbeitung	Vorschläge für Funktionsimplementierung	Kap. 4.2
Gemini 3 Pro	Literaturrecherche	Zusammenfassung aktueller Forschungsarbeiten	Überblick über Stand der Technik	Kap. 2.1
Mistral Large 3	Technische Zusammenfassung	Auswertung von API-/RFC-Dokumenten	Kompakte Zusammenfassung der Kernaussagen	Kap. 2.4
OpenAI GPT-5.2	Formulierungsverbesserung	Überarbeitung der Einleitung	Alternative Formulierungen zur Problemstellung	Kap. 1.2
OpenAI GPT-5.3	Agentische Aufgaben	Recherche zu aktuellen Statistiken	Stichpunkt-Zusammenfassung mit Quellenhinweisen	Kap. 2.3

Bei KI-Tools genau sagen, welches Modell / Version oder ähnliches — frag KI wie man die KI referenziert.

Erklärung der Selbstständigkeit

Hiermit versichere ich, die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt zu haben.

Frankfurt am Main, den 14. März 2026

Leon Rausch